MoCA-Agent: A Market-of-Claims Code Agent for Financial and Numerical Reasoning

Abdelrahman Abdallah[1], **AbdelRahim A. Elmadany**[2], **Sameh Al Natour**[3], **Hasan Cavusoglu**[2], **Adam Jatowt**[1], **Muhammad Abdul-Mageed** [2,λ]
[1]University of Innsbruck [2]University of British Columbia [3]Toronto Metropolitan University
[λ]Canada Research Chair in NLP and ML
muhammad.mageed@ubc.ca

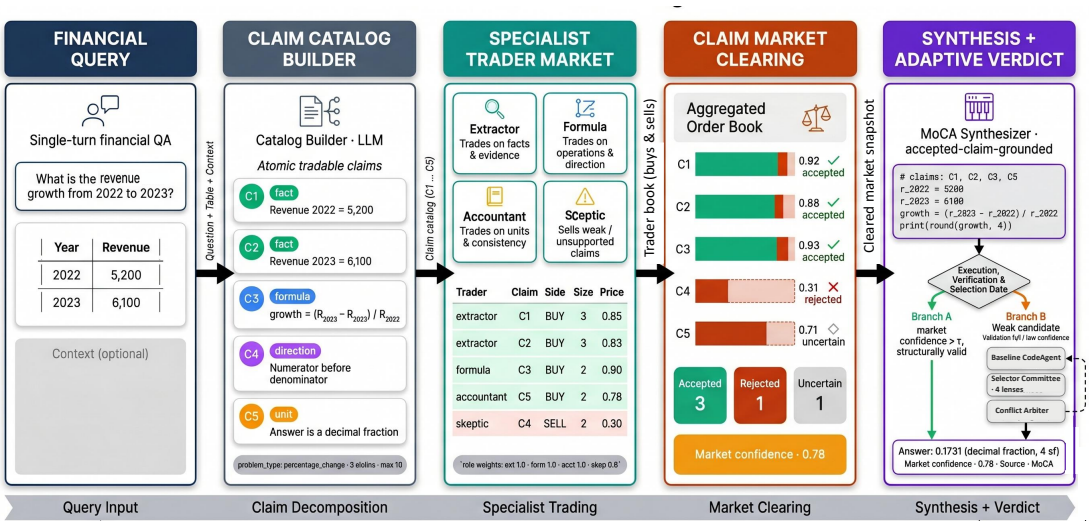


Figure 1: Overview of MoCA-Agent. A claim catalog is built from the question, table, and context. Specialist traders place buy/sell orders on claims; the market clears them into prices, confidences, and statuses. A synthesizer writes a Python program from market-supported claims; an executor runs it, and a verifier validates the output.

Abstract

Financial and tabular question answering requires more than fluent reasoning: answers must be grounded in the exact facts, formulas, units, signs, and scales that support them. A single misread cell or incorrect operation can silently produce a plausible but wrong result. We introduce MOCA-AGENT, a market-of-claims code agent that replaces free-form multi-agent debate with claim-level verification. The system decomposes each question into typed atomic claims, asks specialist trader agents to buy or sell those claims, clears their orders into confidence-weighted accept/reject decisions, and synthesizes an executable Python program from market-supported evidence. A code-aware verifier then checks the program for execution, structural consistency, and common financial reasoning errors, with at most one market-aware repair round. Across ten public benchmarks spanning financial numerical reasoning, general tabular reasoning, ESG question answering, and multimodal chart reasoning, MOCA-AGENT achieves strong performance using a fixed Qwen3.6-27B backbone, including 78.3% on FinQA, 76.0% on FinanceMath, 71.2% on MultiHiertt, 86.9% on ESGenius, and 85.6% average on FinChart-Bench. These results show that aggregating evidence at the level of atomic claims, rather than whole answers, improves robustness in high-stakes numerical reasoning.[1]

1 Introduction

Financial and tabular question answering systems are increasingly being deployed as the analytic backbone of automated reporting, audit, and research-assistant pipelines (Xie et al., 2023, 2024; Krumdick et al., 2024). Yet, unlike open-ended chat tasks, this regime has virtually no tolerance for fluent-but-wrong derivations: a single misread cell, swapped formula, or off-by-100-percent error silently corrupts the final number while the surrounding prose continues to read plausible (Chen et al., 2021; Zhao et al., 2022, 2024b,a). Standard approaches for closing this gap, including chain-

[1]The code and data are available: https://github.com/UBC-NLP/MoCA-Agent.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 10980–11009
October 24-29, 2026 ©2026 Association for Computational Linguistics

of-thought (CoT) (Wei et al., 2022), program-of-thought (PoT) (Chen et al., 2022a; Lu et al., 2023), and free-form multi-agent debate (Du et al., 2024; Madaan et al., 2023; Shinn et al., 2023), address part of the problem. Still, they share a deeper limitation: they operate at the level of complete *outputs* rather than *the atomic claims those outputs depend on*.

Concretely, three failure modes arise in financial code agents: `(I) Silent Miscomputation in PoT`: A PoT agent effectively commits once it emits a program. Even a syntactically valid program can divide the wrong cell by another wrong cell, while downstream consistency checks observe only the printed output rather than the facts, units, signs, and formulas the program assumed. On MultiHiertt (Zhao et al., 2022), OpenAI o1, as reported by Cao et al. (2025), achieves only 38% with textual reasoning and 49% with symbolic reasoning, despite the full table content being provided in the input context. `(II) Opaque Aggregation in Multi-Agent Debate`: Both free-form debate (Du et al., 2024) and critic-actor loops (Shinn et al., 2023; Madaan et al., 2023; Gou et al., 2024) typically aggregate *whole answers* or whole critiques. If multiple agents fluently misderive the same wrong number, the consensus can become wrong with high confidence. Recent multi-agent financial pipelines (Tan et al., 2025; Liu et al., 2025) improve robustness, but they still treat each agent's output as an opaque vote rather than a collection of typed claims that other agents can independently support or challenge. `(III) Insufficient Structural Verification`: Self-refine and code-repair frameworks (Madaan et al., 2023; Shinn et al., 2023; Jin et al., 2025; Bhattarai et al., 2025) also use code or iterative repair. However, their feedback is mainly based on execution failures or free-form critique, making it difficult to detect programs that execute successfully while using the wrong fact, formula, sign, or scale. Common financial mistakes, such as scaling a ratio by 100, flipping the sign on a net-of-tax line, or summing the wrong row total, all execute cleanly. As a result, neither runtime feedback nor free-form critique reliably surfaces the silent-error patterns that dominate financial numerical reasoning.

This work makes four main contributions. **(1) Failure-Mode Analysis and System Design.** We identify three recurring failure modes in financial and tabular code agents: *silent miscomputation*, in which an answer is numerically wrong despite appearing plausible; *opaque aggregation*, where whole-agent outputs are combined without exposing the factual, formulaic, or unit-level assumptions on which they depend; and *insufficient structural verification*, where generated programs are not checked against the arithmetic, accounting, and scale constraints of the task. To address these failures jointly, we propose MoCA-Agent (Figure 1), a code-agent framework that makes these assumptions explicit as typed atomic claims and aggregates them through a structured claim market rather than free-form debate (§3). **(2) Market-of-Claims Protocol.** In this protocol, specialist roles (Extractor, Formula, Accountant, and Skeptic) place weighted, signed orders over typed atomic claims, including facts, formulas, units, signs, and directions of change. The market clears these orders into per-claim prices, confidence scores, and accept/reject/uncertain statuses (§3.4), creating an explicit audit trail of the assumptions underlying the final program. The synthesizer is restricted to non-rejected claims, and the verifier uses the same market record to determine whether the generated program is sufficiently grounded or requires repair. **(3) Code-Aware Verification and Conflict Recovery.** We make the Code-Aware Verifier a core routing component rather than a passive post-hoc sanity check. After executing the synthesized program, the verifier applies operation-aware structural tests that target the most common silent-error patterns in financial reasoning, including sign flips, percent-scale errors, and missing or inconsistent formulas. When the market-grounded program is structurally weak, a Hybrid Selector Committee compares it with an alternative candidate; if the candidates expose unresolved disagreements, a Conflict Arbiter produces a grounded recovery program (§3.5). **(4) Benchmark Results Across Financial and Tabular Tasks.** We evaluate MoCA-Agent on 10 public financial and tabular benchmarks. Using a single Qwen3.6-27B backbone and shared prompting setup, MoCA-Agent matches or surpasses the strongest previously reported results in our comparison on six tracks. The largest gains appear in settings that require both multi-cell extraction and formula composition: $+14.4$ on MultiHiertt over RL-trained Fortune, $+9.0$ on FinanceMath validation over GPT-4o (PoT), $+8.3$ on DocMath-Complong over DeepSeek-V3, and $+4.1$ on FinQA over Fino1-14B (§4).

2 Literature Review

Financial agents have been studied in several forms, from trading and portfolio-management systems to risk-analysis tools and retrieval-based financial question answering. In this work, we focus on a more specific setting: code-producing agents for financial and tabular question answering. These agents must retrieve or extract relevant evidence, reason over tables and text, compose the appropriate calculation, and often generate the executable code for the final answer. We therefore review work along four dimensions: financial and table-reasoning benchmarks, program- and tool-augmented reasoning, multi-agent aggregation, and verification or repair of generated code.

Benchmarks and code-augmented agents. Financial numerical-reasoning datasets such as FinQA (Chen et al., 2021), ConvFinQA (Chen et al., 2022b), TAT-QA (Zhu et al., 2021), Multi-Hiertt (Zhao et al., 2022), and HiTab (Cheng et al., 2022) established text–table reasoning settings that require evidence extraction and program-like numerical derivations. Subsequent work extends this setting to long documents (DocFinQA (Reddy et al., 2024), DocMath-Eval (Zhao et al., 2024b)), harder financial numerical reasoning (FinanceMath (Zhao et al., 2024a), FinanceReasoning (Tang et al., 2025)), and broader financial or business knowledge probes (ESGenius (He et al., 2025), FinBen (Xie et al., 2024), PIXIU (Xie et al., 2023), BizBench (Krumdick et al., 2024)); general-tabular benchmarks include WTQ (Pasupat and Liang, 2015) and TabMWP (Lu et al., 2022). Program-of-thought (Chen et al., 2022a) and chain-of-table (Wang et al., 2024) ground reasoning in executable artifacts, while recent tabular code agents combine plan synthesis with iterative table operations (TableMaster (Cao and Liu, 2025), ReAcTable (Zhang et al., 2024), TIDE-Agent (Yan et al., 2026), ARTEMIS-DA (Hussain, 2024), TableMind (Jiang et al., 2026)). For finance and hierarchical tables, representative strong baselines include Fortune (Cao et al., 2025), TableGPT2 (Su et al., 2024), TabAF (Wang et al., 2025), and Fin-o1 (Qian et al., 2025), with multi-agent extensions in Tan et al. (2025) and Liu et al. (2025). MoCA-Agent instead exposes these assumptions as typed atomic claims before code generation, allowing specialist roles to accept, reject, or challenge the relevant fact, formula, unit, sign, or direction before the final program is synthesized.

Debate, verification, and code repair. Du et al. (2024) show that debate among LLM instances improves factuality, while ReAct (Yao et al., 2022) interleaves reasoning with tool use. Recent work further formalizes deliberation (Wu et al., 2025; Ning et al., 2025; Hu et al., 2026; Zhang and Xiong, 2025), but Wu et al. (2025) note that, under matched compute, debate can reduce to ensembling unless agents disagree over *structured* sub-claims. Self-Refine (Madaan et al., 2023), Reflexion (Shinn et al., 2023), CRITIC (Gou et al., 2024), and code-repair systems (ReVeal (Jin et al., 2025), ARCS (Bhattarai et al., 2025)) iterate on execution signals or LLM-judge critique; these catch crashes but often miss silent financial errors (e.g., sign-flipped net of tax, percent scaled by 100, or ratios with the wrong denominator). In contrast, MoCA-Agent makes structured sub-claims first-class: traders can short an over-confident formula without rejecting the rest of the program, while the verifier derives operation-specific checks from a typed problem class and supplies both runtime and structural feedback to a single market-aware repair round (§3.5).

3 MoCA-Agent

In this section, we present MoCA-Agent, a framework for financial and tabular question answering. Given a question q, an optional table T, and free-form context C, MoCA-Agent produces an executable Python program $\mathcal{P}$, whose printed output is returned as the final answer a. Unlike free-form multi-agent debate systems, MoCA-Agent grounds the program derivation in *market-cleared atomic claims*. By making the assumptions behind the program explicit, MoCA-Agent makes the reasoning process traceable and reduces the risk that a mis-extracted cell, incorrect formula, or scale error silently propagates through the pipeline as if it were an accepted fact.

Figure 1 shows the full pipeline. First, a Catalog Builder decomposes $\langle q, T, C \rangle$ into atomic, tradable *claims* $\mathcal{K} = \{k_1, \ldots, k_M\}$ covering the facts, formulas, units, signs, and *directions of change* (e.g., increase vs. decrease) needed to answer the question. Second, four Specialist Trader Agents $\mathcal{R}$ (EXTRACTOR, FORMULA, ACCOUNTANT, SKEPTIC) independently buy or sell each claim, expressing role-specific confidence through price and notional size. Third, a Claim Market clears these orders into per-claim prices π_m, confidence scores

γ_m, and accept/reject/uncertain statuses. Fourth, a Synthesizer writes a Python program $\mathcal{P}$ using only non-rejected market claims. Finally, a Code-Aware Verifier executes $\mathcal{P}$, checks the output against problem-specific structural rules, and triggers at most one Market-Aware Repair round if validation fails. When the market-grounded candidate is weak, a Hybrid Selector Committee may instead promote a baseline candidate; a Conflict Arbiter handles remaining disagreements.

3.1 Query

MOCA-AGENT takes as input a tuple $\langle q, T, C\rangle$, where q is a natural-language question, T is an optional table, and C is free-form supporting context. The table may be text-only, hierarchical, or transcribed from a chart by a VLM in the multimodal setting; see §3.6. MOCA-AGENT returns an executable Python program $\mathcal{P}$ whose printed output is the final answer a. All downstream stages operate on this tuple, together with the typed claim catalog produced in §3.2.

3.2 Claim Catalog Builder

The catalog builder receives $\langle q, T, C\rangle$ and returns a typed catalog

$$\mathcal{K} = \{k_m = (id_m,\, \kappa_m,\, \sigma_m,\, \nu_m,\, \epsilon_m)\}_{m=1}^{M}, \quad (1)$$

where σ_m is a one-line natural-language summary, ν_m is the candidate value (a number, an expression, or a unit token), and ϵ_m is a short evidence quote. The claim kind κ_m takes one of six labels: *fact*, *formula*, *unit*, *sign*, *direction*, or *other*. The direction label captures operation polarity such as increase/decrease or numerator/denominator orientation. We cap the number of claims at $M_{\max} = 10$ to keep the downstream market tractable. The catalog also tags the question with a *problem type* τ, drawn from *percentage-change*, *ratio*, *sum*, *difference*, *average*, *comparison*, or *other*, which the verifier later uses to select operation-aware checks.

3.3 Specialist Trader Market

Each specialist trader $r \in \mathcal{R}$ reads $\langle q, T, C, \mathcal{K}\rangle$ and returns an order book $\mathcal{B}_r = \{(id_m, s, n, p, \rho)\}$, where $s \in \{\text{buy}, \text{sell}\}$, the size $n \in \{1, \ldots, 5\}$, the price $p \in [0.01, 0.99]$, and ρ is a short rationale. Buy orders indicate support for a claim; sell orders indicate doubt. Each role has a complementary inductive bias: EXTRACTOR grounds factual cells against the table, FORMULA validates that proposed formulas obey the requested operation, ACCOUNTANT cross-checks units and accounting conventions (e.g., revenue, expense, net), and SKEPTIC actively sells under-evidenced claims.

3.4 Claim Market Clearing

For each claim k_m, the market aggregates weighted notional volume across all roles. Let $w_r \geq 0$ denote the weight assigned to role r. In our implementation, $w_{\text{ACCOUNTANT}}$=1.10, w_{SKEPTIC}=0.95, and $w_{\text{EXTRACTOR}}$=w_{FORMULA}=1.00. Define

$$B_m = \sum_{\substack{r,\,(id_m,\text{buy},n,p,\cdot)\\ \in\mathcal{B}_r}} w_r\, n\, p, \quad (2)$$

$$S_m = \sum_{\substack{r,\,(id_m,\text{sell},n,p,\cdot)\\ \in\mathcal{B}_r}} w_r\, n\, p. \quad (3)$$

The market clears claim k_m at price

$$\pi_m \;=\; \frac{B_m}{B_m + S_m + \varepsilon}, \quad (4)$$

with confidence

$$\gamma_m \;=\; \frac{|B_m - S_m|}{B_m + S_m + \varepsilon}, \quad (5)$$

where $\varepsilon > 0$ is a small numerical stabilizer that prevents division by zero for claims with no orders (we use $\varepsilon = 10^{-6}$). The market then assigns each claim a status

$$\zeta_m = \begin{cases} \text{accepted} & \pi_m \geq \pi^{\uparrow}, \\ \text{rejected} & \pi_m \leq \pi^{\downarrow}, \\ \text{uncertain} & \text{otherwise.} \end{cases} \quad (6)$$

We use $\pi^{\uparrow}$=0.62, $\pi^{\downarrow}$=0.38. Intuitively, π_m is the fraction of weighted volume that buys the claim, and γ_m is the magnitude of the disagreement-normalized imbalance. For any nonempty subset of claim ids $\mathcal{U}$, we define the program-level market confidence as

$$\Gamma(\mathcal{U}) \;=\; \frac{1}{|\mathcal{U}|} \sum_{m\in\mathcal{U}} \gamma_m. \quad (7)$$

This value is used by the verifier and selector.

3.5 Synthesis and Adaptive Verdict

This stage turns the cleared market into a final executable program. It has three components: a *synthesizer* that drafts a Python program restricted to non-rejected market claims; a *code-aware verifier* that

runs the program and applies operation-aware structural checks, triggering at most one *market-aware repair* round on failure; and a *hybrid selector committee with a conflict arbiter* that compares against a baseline candidate when the market-grounded program is structurally weak.

Synthesizer. The synthesizer receives $\langle q, T, C, \mathcal{K}, \{(\pi_m, \gamma_m, \zeta_m)\}\rangle$ and returns

$$\mathcal{P} = (\textit{reasoning}, \mathcal{U}, \textit{abstain}, \textit{code}, a), \quad (8)$$

where $\mathcal{U} \subseteq \mathcal{K}$ is the set of claims actually used, and $m \in \mathcal{U}$ refers to the index of claim $k_m \in \mathcal{U}$. The key constraint is simple: the synthesizer may only cite claims with $\zeta_m \in \{\text{accepted}, \text{uncertain}\}$. Rejected claims are filtered out before the program is written. For multimodal or open-form questions, an upstream *plan-then-execute* reasoner produces a prose derivation that is appended to the synthesizer prompt; the same claim-grounding constraint still applies.

Code-Aware Verifier and Market-Aware Repair. The executor runs $\mathcal{P}$ in a 10-second sandbox and returns $(\textit{success}, \textit{out}, \textit{err}, t)$. Each violation contributes -0.45 to $\Phi(\mathcal{P})$; a fact-grounded, operation-matched, market-confident program is rewarded $+1.00+0.20\min(|\mathcal{U}_{\text{accepted}}|, 4)+0.40\,\Gamma(\mathcal{U})$. A candidate is accepted when $\Phi(\mathcal{P}) \geq 2.20$ (the threshold used in our selector, §3.5). (i) that `success` is true; (ii) that the cited claims include enough *fact* claims for the operation (e.g., ≥ 2 when τ is one of percentage-change, ratio, sum, difference, average, or comparison); (iii) that a formula/unit/sign claim is present when τ requires it; (iv) that none of the cited claims were rejected by the market; (v) that the printed answer is consistent with question polarity (yes/no question $\Rightarrow$ yes/no output, percent question without an explicit "percent" scale tag does not multiply by 100, etc.); (vi) that the operation actually appears in the source code (e.g., "ratio" implies a division, "difference" a subtraction). Each violation contributes -0.45 to the candidate score; a fact-grounded, operation-matched, market-confident program is rewarded $+1.00 + 0.20\cdot\min(|\mathcal{U}_{\text{accepted}}|, 4) + 0.40\,\Gamma(\mathcal{U})$.

If the candidate is invalid, market-aware repair runs once. The repair prompt receives both the execution error and the verifier's structural reason list (e.g., *missing_formula_or_unit_claim*, *percent_scaled_by_100*), so the synthesizer can target the specific defect rather than rewriting the program without guidance.

Hybrid Selector Committee and Conflict Arbiter. When the market-grounded candidate is structurally weak, meaning the verifier score $\Phi(\mathcal{P})$ is below 2.20 or market confidence $\Gamma(\mathcal{U})$ is below 0.78[2], MOCA-AGENT also calls a *baseline proposer*, a single-agent writer with no claim market. A four-aspect committee (GENERAL, EXTRACTION, FORMULA, SCALE) compares the two candidates side by side. If the committee's votes are split between the two candidates, a *conflict arbiter* writes a fresh third program that draws explicitly on the parts of each candidate the committee approved. The router keeps the highest-scoring grounded candidate; in our default configuration the baseline proposer is bypassed for open-ended / symbolic answer formats, because its numeric-only prompt would overwrite non-numeric outputs.

3.6 Multimodal Extension

For multimodal benchmarks, such as FinChart-Bench (Shu et al., 2025), we make only one change: the empty table slot T is filled with a question-conditioned vision-language transcription of the supplied chart, produced by a single VLM call. The trader market, synthesizer, executor, and verifier are otherwise identical to the text-only configuration. This keeps the evaluation focused on the agent pipeline rather than on a stronger vision encoder.

4 Experiments

4.1 Evaluation Datasets

We evaluate MOCA-AGENT on ten public benchmarks across four categories; full details, splits, and statistics are in Appendix A. (C1) Financial Numerical Reasoning covers reports, tables, long documents, and expert-written finance programs via *FinQA* (Chen et al., 2021), *DocMath-Simplong / DocMath-Complong* (Zhao et al., 2024b), and *FinanceMath* (Zhao et al., 2024a). (C2) General Tabular Reasoning spans hierarchical, multi-table, mathematical, and open-domain tables via *HiTab* (Cheng et al., 2022), *MultiHiertt* (Zhao et al., 2022), *TabMWP* (Lu et al., 2022), and *WikiTableQuestions* (Pasupat and Liang, 2015). (C3) Domain Knowledge QA uses *ESGenius* (He et al., 2025), an expert-validated ESG multiple-choice benchmark, under both zero-shot and RAG protocols. (C4) Multimodal Chart Reasoning uses

[2]Both thresholds were calibrated on a 100-question held-out development split from FinQA dev and then frozen across all ten benchmarks; no per-dataset tuning.

Method	TabMWP	WTQ	HiTab	MultiHrt
Fine-tuning approaches				
TAPEX-Large (Liu et al., 2021)	–	59.10	45.60	–
OmniTab (Jiang et al., 2022)	–	62.80	–	–
TableLLM (Zhang et al., 2025)	–	53.59	43.88	–
TableGPT2 (Su et al., 2024)	–	61.42	70.27	–
TabAF (Wang et al., 2025)	–	74.72	78.41	–
NAPG (Zhang et al., 2023)	–	–	–	44.19
Prompting / in-context learning				
PoT (Lu et al., 2023)	89.49	–	–	–
CoT (Lu et al., 2023)	90.81	–	–	–
Chain-of-Table (Wang et al., 2024)	–	67.31	–	–
SS-CoT (Shi et al., 2025)	–	76.80	79.10	–
TableMaster (Cao and Liu, 2025)	–	78.13	–	–
ARTEMIS-DA (Hussain, 2024)	–	80.80	–	–
Tool-augmented / agent-based				
CRITIC (Gou et al., 2024)	89.00	–	–	–
PoT+Doc (Hsieh et al., 2023)	92.69	–	–	–
Chameleon (Lu et al., 2023)	93.28	–	–	–
CREATOR (Qian et al., 2023)	94.70	–	–	–
Reinforcement learning				
RL w/ CS (Formula) (Cao et al., 2025)	–	–	–	56.78
MoCA-Agent (ours)	**96.00**	**81.40**	77.27	**71.17**

Table 1: **General Tabular Reasoning.** Accuracy (%) on TabMWP, WTQ, HiTab test and MultiHiertt dev. baselines use varied backbones (BART-Large to GPT-4). See Appendices B.3 and B.6 for full per-dataset leaderboards. Underline marks the best previously published result; bold marks the best overall result.

FinChart-Bench (Shu et al., 2025), with True/False, multiple-choice, and open QA subtasks; we report all three and emphasize open QA, the hardest setting. For each benchmark, we compare against the established state-of-the-art results reported in the original benchmark paper(s), summarized in Tables 2- 6.

4.2 Evaluation

We follow the official evaluation protocol of each dataset and report *Exact Match* or *Execution Accuracy* as the primary performance metric, depending on the benchmark specification. In addition to task-level accuracy, we track three internal diagnostics for MoCA-Agent: *Code Execution Rate,* defined as the proportion of generated programs that execute without error; *Code–Answer Consistency,* which measures whether the printed program output matches the reported answer; and *Mean Market Confidence* $\bar{\Gamma}$, which summarizes the system's aggregate confidence after market clearing. These diagnostics are reported in Appendix B.9, Table B.9.

Method	FinQA	DM-S	DM-C	FinMath
Proprietary LLMs				
GPT-4o (Hurst et al., 2024)	72.49	60.00	39.33	67.0
GPT-o1-preview	49.07	56.00	36.67	–
GPT-o3-mini	60.87	59.00	35.00	–
GPT-4.5	68.94	59.00	39.33	–
DeepSeek-V3 (Liu et al., 2024)	73.20	53.00	42.33	–
DeepSeek-R1 (Guo et al., 2025)	65.13	53.00	38.67	–
Claude-3.5-Sonnet	–	–	–	60.6
Open-source LLMs				
Llama-3.3-70B	68.15	54.00	32.00	–
Qwen2.5-72B	73.38	59.00	14.67	–
DS-R1-Distill-Llama-70B	66.73	53.00	30.67	–
Qwen2.5-32B	73.11	56.00	30.00	–
Qwen3-32B	64.15	51.00	26.00	–
Fino1-7B	73.03	56.00	26.33	–
Fino1-14B	74.18	55.00	27.33	–
MoCA-Agent	**78.29**	**70.00**	**50.67**	**76.00**

Table 2: **Financial numerical reasoning.** Exact-match or execution accuracy (%). DM-S/DM-C denote DocMath-Simplong/Complong; FinMath denotes FinanceMath validation. Baselines are from (Zhao et al., 2024b,a). Underline marks the best previously published result; bold marks the best overall result.

4.3 Experimental Setup

All MoCA-Agent results in the main tables use the same Qwen3.6-27B backbone served via vLLM (OpenAI-compatible) for every role: trader panel, synthesizer, verifier LLM calls, and selector committee. The prompts are the same across datasets; only the loader changes. We use $M_{\max}$=10 claims, four roles with the weights from §3.3, market thresholds $(\pi^{\uparrow}, \pi^{\downarrow})$=$(0.62, 0.38)$, one repair round, and a 10 s sandbox time-out. The hybrid baseline proposer is enabled with selector thresholds $(\text{score}, \Gamma) \geq (2.20, 0.78)$ and arbiter min-score 2.25.

5 Results and Discussion

C1: Financial Numerical Reasoning. Table 2 reports results on FinQA, DocMath-Simplong/Complong, and FinanceMath. MoCA-Agent achieves 78.29% on FinQA, improving over Fino1-14B (74.18%) by +4.11 points. The largest gains are on long-document reasoning: 70.0% on DM-Simplong (+10.0 over GPT-4o) and 50.7% on DM-Complong (+8.34 over DeepSeek-V3). On FinanceMath, MoCA-Agent reaches 76.0%, outperforming GPT-4o PoT (67.0%) by +9.0 points. Two characteristics of this category make the market-of-claims design useful: (i) answers often hinge on a small set of cells that must each be extracted correctly, which the Extractor role concentrates its trading volume on; and (ii) er-

ror modes often involve sign / unit / scale slips, which the verifier's operation-aware checks catch before the program is committed.

Method	Setting	ESGenius
Best zero-shot baselines (He et al., 2025)		
GPT-4o	ZS	63.64
o4-mini	ZS	69.45
o3	ZS	72.54
Best RAG baselines (He et al., 2025)		
Qwen2.5-72B-Instruct + RAG	RAG	82.57
Gemma-3 12B + IT + RAG	RAG	83.80
Qwen2.5-32B-Instruct + IT + RAG	RAG	82.47
MoCA-Agent (ours)	Agent	**86.88**

Table 3: **Domain Knowledge QA on ESGenius.** MoCA-Agent uses a claim-market agent rather than zero-shot prompting or a static RAG retriever.

C2: General Tabular Reasoning. Table 1 reports results on the four general-tabular benchmarks. MoCA-Agent achieves the best score on three of four splits: 96.00% on TabMWP, improving over CREATOR by +1.30 points; 81.40% on WTQ, improving over ARTEMIS-DA/GPT-4 by +0.60 points; and 71.17% on MultiHiertt, outperforming Fortune RL w/ CS Formula (56.78%) by +14.39 points. On HiTab, MoCA-Agent reaches 77.27%, remaining close to the published SS-CoT result, with a gap of −1.83 points. Overall, these results show that MoCA-Agent generalizes well beyond financial tables, particularly in settings that require multi-step reasoning over hierarchical or open-domain tabular structures.

C3: Domain Knowledge QA. On ESGenius (Table 3), MoCA-Agent reaches 86.88%, +3.08 over the strongest published RAG system (Gemma-3 12B + IT + RAG at 83.80%) and +14.34 over the strongest zero-shot system (o3 at 72.54%). The gain over zero-shot is expected because ESGenius rewards retrieval, but MoCA-Agent also outperforms every published RAG configuration despite using *no* external retriever; instead, the claim catalog routes the question into typed factual subclaims that the trader panel scrutinizes, and the SKEPTIC role suppresses confidently wrong distractors that hurt the zero-shot baselines.

C4: Multimodal Chart Reasoning. Table 6 reports results across the three FinChart-Bench subtasks. MoCA-Agent is the strongest system overall, with an 85.59% three-subtask average that beats the best closed-source baseline (Claude Sonnet 4, 84.32%) by +1.27 points.

Agent	FinQA	MultiHiertt
All agents on Qwen3.6-27B (same backbone)		
TradingAgents (Xiao et al., 2024)	74.50	63.70
FINCON (Yu et al., 2024)	73.20	62.00
SheetBrain (Wang et al., 2026)	71.20	62.60
MoCA-Agent (ours)	**78.29**	**71.17**
Δ over best competing agent	+3.79	+7.47

Table 4: Head-to-head comparison at *fixed* backbone (Qwen3.6-27B). Bold makes best at fixed backbone.

The headline result is on the open QA subtask, where MoCA-Agent reaches 74.65% and improves over Claude Sonnet 4 (63.59%) by +11.06 points. Open numerical questions over chart-extracted figures are precisely the setting where a single-pass VLM tends to state a plausible answer while silently using the wrong value from the chart; in MoCA-Agent, such values are shorted by the ACCOUNTANT and SKEPTIC roles before any program is written, which prevents the silent miscomputation from propagating to the final answer. On the other two subtasks, MoCA-Agent is competitive but not the absolute best. On MC (84.85%) it leads every open-source baseline, including Mistral 3.1 24B (+0.68), and surpasses two closed-source systems (GPT-4o, GPT-4.1), while trailing the strongest five closed-source VLMs by 1.9–7.3 points. On TF (97.27%) it ties Gemini 2.5 Pro and trails o3 and o4-mini by less than one point. Notably, this is the only category in the paper that exercises a multimodal pipeline, showing that the agent design transfers from text-only tables to chart transcriptions without modification.

5.1 Comparison with Other Agentic Methods

The benchmark tables in §5 compare MoCA-Agent against published numbers from heterogeneous backbones (GPT-4o, Claude Sonnet 4, DeepSeek-V3, Llama-3.1-70B, Qwen2.5-Coder-7B), making it hard to separate agent design from model quality. To isolate the agent effect, we re-implement three recent finance/table agents on the same Qwen3.6-27B backbone with identical decoding and loaders: *TradingAgents* (Xiao et al., 2024), *FINCON* (Yu et al., 2024), and *SheetBrain* (Wang et al., 2026), on FinQA and MultiHiertt (the two with runnable releases for all three). With the back-

Backbone	FinQA	DM-Simp	DM-Comp	FinMath	HiTab	MultiHrt	TabMWP	WTQ	ESGenius	FinChart-QA	Avg.
Qwen3.6-27B (default)	**78.29**	**70.00**	**50.67**	**76.00**	77.27	**71.17**	**96.00**	81.40	**86.88**	**74.65**	**76.23**
Gemma-4 31B Instruct	77.86	68.00	45.67	73.50	**77.34**	67.15	93.60	**82.98**	85.12	72.52	74.77
Qwen-3.5 9B	75.31	67.65	42.00	69.19	73.14	65.87	91.87	73.16	85.30	72.69	71.62

Table 5: Backbone scaling for MOCA-AGENT. Each row is the full agent (byte-identical pipeline) served on a different backbone. DM-Simp/DM-Comp denotes DocMath Simplong/Complong; FinMath denotes FinanceMath validation; MultiHrt denotes MultiHiertt dev.

Method	Backbone	TF	MC	QA	Avg.
Open-source VLMs (Shu et al., 2025)					
LLaVa v1.6	LLaVa v1.6 (7B)	63.30	36.94	1.84	34.47
DeepSeek VL2	DeepSeek VL2 (2.8B)	57.97	6.77	4.73	23.50
LLaMa 3.2	LLaMa 3.2 (11B)	63.46	64.00	5.12	44.65
Cosmos	Cosmos (7B)	70.93	43.49	20.18	45.22
Sa2VA	Sa2VA (8B)	86.62	75.79	19.43	61.12
Qwen2 VL	Qwen2 VL (7B)	91.86	19.40	24.77	45.76
Gemma 3	Gemma 3 (12B)	89.14	70.94	27.22	62.89
Qwen2.5 VL	Qwen2.5 VL (7B)	95.09	82.81	37.29	72.16
Mistral 3.1	Mistral 3.1 (24B)	92.95	84.17	44.90	74.37
LLaMa 4	LLaMa 4 (17B)	89.56	59.70	59.78	69.89
Closed-source VLMs (Shu et al., 2025)					
GPT 4o	GPT 4o	93.92	77.57	57.59	76.62
GPT 4.1 mini	GPT 4.1 mini	91.61	86.77	60.30	79.80
GPT 4.1	GPT 4.1	96.18	83.19	62.54	80.88
o4 mini	o4 mini	97.61	91.96	60.18	83.53
Gemini 2.5 Pro	Gemini 2.5 Pro	97.27	89.70	63.46	83.73
o3	o3	97.86	92.17	60.79	83.89
Claude Sonnet 4	Claude Sonnet 4	96.94	91.66	63.59	84.32
MOCA-AGENT (ours)	Qwen3.6-27B + VLM	97.27	84.85	**74.65**	**85.59**

Table 6: **Multimodal Chart Reasoning** on FinChart across the three subtasks: True/False (TF), Multiple-Choice (MC), and open Question-Answering (QA).

Configuration	FinQA	MultiHiertt	Avg.
Full MOCA-AGENT	**78.29**	**71.17**	**74.73**
w/o Trader Market	73.38 (−4.91)	68.50 (−2.67)	70.94 (−3.79)
w/o Code Repair	74.50 (−3.79)	69.00 (−2.17)	71.75 (−2.98)
w/o Baseline Proposer	72.00 (−6.29)	67.20 (−3.97)	69.60 (−5.13)
w/o Conflict Arbiter	75.50 (−2.79)	68.30 (−2.87)	71.90 (−2.83)

Table 7: Component ablation at fixed Qwen3.6-27B backbone, with Δ relative to Full MOCA-AGENT in parentheses. Accuracy (%) on the official splits used in Tables 2 and 1.

bone fixed (Table 4), MOCA-AGENT exceeds the strongest competitor by +3.79 on FinQA (78.29 vs. 74.50) and +7.47 on MultiHiertt (71.17 vs. 63.70), both held by TradingAgents. The MultiHiertt gap is informative: TradingAgents and FINCON aggregate whole answers via debate or conceptual reinforcement, and SheetBrain operates cell-by-cell without a typed claim abstraction. None can short an over-confident formula independently of the rest of the program — exactly what the trader market provides, and exactly the failure mode hierarchical-table joining exposes.

5.2 Backbone Scaling

To check whether the gains depend on Qwen3.6-27B, we re-run MOCA-AGENT on Gemma-4 31B Instruct (cross-family, comparable capacity) and Qwen-3.5 9B (same-family, smaller) with identical prompts and decoding. MOCA-AGENT transfers across both axes. Gemma-4-31B reaches 74.77 average (−1.46) and leads on HiTab (77.34 vs. 77.27) and WTQ (82.98 vs. 81.40), showing the trader market is not Qwen-specific. Qwen-3.5 9B drops 4.61 on average, but the loss concentrates on the hardest long-form settings (DM-Complong −8.67, FinanceMath −6.81, WTQ −8.24) and stays small on FinQA (−2.98), FinChart-QA (−1.96), and ESGenius (−1.58), indicating the mechanism preserves most of its benefit at 9B. The gains in Tables 2–6 are attributable to the agent, not the base LLM.

5.3 Component Ablation

We disable one component at a time on FinQA and MultiHiertt: the four-role TRADER MARKET (replaced by a single FORMULA trader), CODE REPAIR, the BASELINE PROPOSER, and the CONFLICT ARBITER. Each removal drops the two-benchmark average by 2.8–5.1 points, confirming they are complementary. BASELINE PROPOSER contributes most (−5.13 Avg.): even a strong market-grounded program benefits from a second opinion. TRADER MARKET is next (−3.79) and matters more on FinQA (−4.91) than MultiHiertt (−2.67), since FinQA's regular tables put the burden on extraction while MultiHiertt's hierarchical joins are already caught by the verifier. CODE REPAIR and CONFLICT ARBITER show similar drops (−2.98, −2.83) but opposite tilts: repair helps more on FinQA, the Arbiter more on MultiHiertt — consistent with MultiHiertt inducing program-level disagreements while FinQA's shorter programs are usually salvaged in one repair pass.

6 Conclusion

We presented MOCA-AGENT, a code-agent framework that replaces free-form multi-agent debate with a structured market of claims. It decomposes each problem into typed atomic claims over facts, formulas, units, signs, and operation directions, asks specialist traders to support or challenge them, and constrains the synthesizer to use only non-rejected market claims. Across financial, tabular, domain-knowledge, and multimodal chart benchmarks, MOCA-AGENT shows strong performance where precise evidence extraction and formula composition matter most. Head-to-head comparisons, backbone scaling, and ablations attribute the gains to the interaction between claim-level aggregation, code-aware verification, market-aware repair, and conflict resolution rather than the backbone alone.

Limitations

MOCA-AGENT has four notable limitations.

- **Cost.** A full pipeline call consumes six to ten LLM calls (catalog, four traders, synthesizer, optional repair, optional baseline proposer, optional committee), roughly $5\times$ the cost of a single-pass PoT agent; this is comparable to one round of free-form multi-agent debate but substantially more than a single-shot prompt.

- **Out-of-distribution table layouts.** The catalog builder assumes the question can be decomposed into ≤ 10 atomic claims; on benchmarks where the dominant operation is in-distribution column-header normalisation rather than silent miscomputation (notably HiTab, our only sub-SOTA result), the market mechanism does not recover the gap to fine-tuned table specialists.

- **Single multimodal step.** For FinChart-Bench we use a single VLM transcription call before invoking the text-only market; errors in that transcription are not currently re-traded by the panel, so the agent inherits any silent misreads of axis labels or legend entries.

- **English, finance-style evaluation.** All ten benchmarks are English, and the role inductive biases (ACCOUNTANT, SKEPTIC) and verifier checks (percent scaling, sign flips, units) are calibrated to finance / accounting conventions; transfer to scientific or biomedical numerical reasoning, or to non-English financial corpora, is not validated in this work.

Ethical Considerations

MOCA-AGENT is designed for financial and numerical question answering, a setting where errors may affect reporting, audit, research, or decision-support workflows. Although the proposed market-of-claims mechanism improves traceability by exposing which facts, formulas, units, signs, and directions are accepted or rejected, it does not eliminate the risk of incorrect answers. Market confidence should therefore not be interpreted as a guarantee of truth or as calibrated financial certainty. Outputs from the system should be treated as decision-support artifacts and reviewed by qualified humans before being used in high-stakes financial, legal, regulatory, or investment contexts.

The experiments in this paper use public benchmarks and do not introduce a new dataset of private financial records or personal information. However, practical deployment on proprietary documents could raise privacy and confidentiality concerns. Any such use should ensure that sensitive financial data are handled under appropriate access controls, logging policies, and data-governance procedures.

The system may also be misused to produce persuasive but incorrect financial summaries or to automate unsupported claims about companies, markets, or ESG performance. To reduce this risk, deployed versions should preserve the claim-level audit trail, expose rejected or uncertain claims, and clearly distinguish verified computations from model-generated interpretations.

Acknowledgments

Muhammad Abdul-Mageed acknowledges support from Canada Research Chairs (CRC), the Natural Sciences and Engineering Research Council of Canada (NSERC; RGPIN-2026-07098), the Social Sciences and Humanities Research Council of Canada (SSHRC; 895-2020-1004), Canadian Foundation for Innovation (CFI; 37771), Digital Research Alliance of Canada,[3] and UBC ARC-Sockeye.

References

Manish Bhattarai, Miguel Cordova, Minh Vu, Javier Santos, Ismael Boureima, and Dan O'Malley. 2025.

[3] https://alliancecan.ca

Arcs: Agentic retrieval-augmented code synthesis with iterative refinement.

Lang Cao and Hanbing Liu. 2025. Tablemaster: A recipe to advance table understanding with language models. *arXiv preprint arXiv:2501.19378*.

Lang Cao, Jingxian Xu, Hanbing Liu, Jinyu Wang, Mengyu Zhou, Haoyu Dong, Shi Han, and Dongmei Zhang. 2025. Fortune: Formula-driven reinforcement learning for symbolic table reasoning in language models. *arXiv preprint arXiv:2505.23667*.

Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. 2022a. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*.

Zhiyu Chen, Wenhu Chen, Charese Smiley, Sameena Shah, Iana Borova, Dylan Langdon, Reema Moussa, Matt Beane, Ting-Hao Huang, Bryan R Routledge, and 1 others. 2021. Finqa: A dataset of numerical reasoning over financial data. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 3697–3711.

Zhiyu Chen, Shiyang Li, Charese Smiley, Zhiqiang Ma, Sameena Shah, and William Yang Wang. 2022b. Convfinqa: Exploring the chain of numerical reasoning in conversational finance question answering. In *Proceedings of the 2022 conference on empirical methods in natural language processing*, pages 6279–6292.

Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. 2022. Hitab: A hierarchical table dataset for question answering and natural language generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1094–1110.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In *Forty-first international conference on machine learning*.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yujiu Yang, Nan Duan, Weizhu Chen, and 1 others. 2024. Critic: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, volume 2024, pages 57734–57811.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.

Chaoyue He, Xin Zhou, Yi Wu, Xinjia Yu, Yan Zhang, Lei Zhang, Di Wang, Shengfei Lyu, Hong Xu, Wang Xiaoqiao, and 1 others. 2025. Esgenius: Benchmarking llms on environmental, social, and governance (esg) and sustainability knowledge. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 14623–14664.

Cheng-Yu Hsieh, Si-An Chen, Chun-Liang Li, Yasuhisa Fujii, Alexander Ratner, Chen-Yu Lee, Ranjay Krishna, and Tomas Pfister. 2023. Tool documentation enables zero-shot tool-usage with large language models. *arXiv preprint arXiv:2308.00675*.

Tianyu Hu, Zhen Tan, Song Wang, Huaizhi Qu, and Tianlong Chen. 2026. Multi-agent debate for llm judges with adaptive stability detection. *Advances in Neural Information Processing Systems*, 38:46504–46540.

Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, and 1 others. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276*.

Atin Sakkeer Hussain. 2024. Artemis-da: an advanced reasoning and transformation engine for multi-step insight synthesis in data analytics. *arXiv preprint arXiv:2412.14146*.

Chuang Jiang, Mingyue Cheng, Xiaoyu Tao, Qingyang Mao, Jie Ouyang, and Qi Liu. 2026. Tablemind: An autonomous programmatic agent for tool-augmented table reasoning. In *Proceedings of the Nineteenth ACM International Conference on Web Search and Data Mining*, WSDM '26, page 260–270, New York, NY, USA. Association for Computing Machinery.

Zhengbao Jiang, Yi Mao, Pengcheng He, Graham Neubig, and Weizhu Chen. 2022. Omnitab: Pretraining with natural and synthetic data for few-shot table-based question answering. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 932–942.

Yiyang Jin, Kunzhao Xu, Hang Li, Xueting Han, Yanmin Zhou, Cheng Li, and Jing Bai. 2025. Reveal: Self-evolving code agents via reliable self-verification.

Michael Krumdick, Rik Koncel-Kedziorski, Viet Dac Lai, Varshini Reddy, Charles Lovering, and Chris Tanner. 2024. BizBench: A quantitative reasoning benchmark for business and finance. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8309–8332, Bangkok, Thailand. Association for Computational Linguistics.

Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, and 1 others. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.

Chenghao Liu, Qian Liu, Ziqin Zhu, Hao Fei, and Aniket Mahanti. 2025. David vs. goliath: Cost-efficient financial QA via cascaded multi-agent reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 4212–4229, Suzhou, China. Association for Computational Linguistics.

Qian Liu, Bei Chen, Jiaqi Guo, Morteza Ziyadi, Zeqi Lin, Weizhu Chen, and Jian-Guang Lou. 2021. Tapex: Table pre-training via learning a neural sql executor. *arXiv preprint arXiv:2107.07653*.

Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-play compositional reasoning with large language models. *Advances in Neural Information Processing Systems*, 36:43447–43478.

Pan Lu, Liang Qiu, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, Tanmay Rajpurohit, Peter Clark, and Ashwin Kalyan. 2022. Dynamic prompt learning via policy gradient for semi-structured mathematical reasoning. *arXiv preprint arXiv:2209.14610*.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, and 1 others. 2023. Self-refine: Iterative refinement with self-feedback. *Advances in neural information processing systems*, 36:46534–46594.

Yucheng Ning, Xixun Lin, Fang Fang, and Yanan Cao. 2025. Mad-fact: A multi-agent debate framework for long-form factuality evaluation in llms. *arXiv preprint arXiv:2510.22967*.

Panupong Pasupat and Percy Liang. 2015. Compositional semantic parsing on semi-structured tables. In *Proceedings of the 53rd Annual Meeting of the Association for Computational Linguistics and the 7th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 1470–1480.

Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. Creator: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6922–6939.

Lingfei Qian, Weipeng Zhou, Yan Wang, Xueqing Peng, Han Yi, Yilun Zhao, Jimin Huang, Qianqian Xie, and Jian-yun Nie. 2025. Fino1: On the transferability of reasoning-enhanced llms and reinforcement learning to finance. *arXiv preprint arXiv:2502.08127*.

Varshini Reddy, Rik Koncel-Kedziorski, Viet Dac Lai, Michael Krumdick, Charles Lovering, and Chris Tanner. 2024. Docfinqa: A long-context financial reasoning dataset. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 445–458.

Jihao Shi, Xiao Ding, Hengwei Zhao, Ting Liu, and Bing Qin. 2025. Bridging neural and symbolic reasoning: A dual-system framework for interpretable question answering. In *ICASSP 2025-2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.

Dong Shu, Haoyang Yuan, Yuchen Wang, Yanguang Liu, Huopu Zhang, Haiyan Zhao, and Mengnan Du. 2025. Finchart-bench: Benchmarking financial chart comprehension in vision-language models. *arXiv preprint arXiv:2507.14823*.

Aofeng Su, Aowen Wang, Chao Ye, Chen Zhou, Ga Zhang, Gang Chen, Guangcheng Zhu, Haobo Wang, Haokai Xu, Hao Chen, and 1 others. 2024. Tablegpt2: A large multimodal model with tabular data integration. *arXiv preprint arXiv:2411.02059*.

Nelvin Tan, Zian Seng, Liang Zhang, Yu-Ching Shih, Dong Yang, and Amol Salunkhe. 2025. Improved llm agents for financial document question answering. *arXiv preprint arXiv:2506.08726*.

Zichen Tang, Haihong E, Ziyan Ma, Haoyang He, Jiacheng Liu, Zhongjun Yang, Zihua Rong, Rongjin Li, Kun Ji, Qing Huang, Xinyang Hu, Yang Liu, and Qianhe Zheng. 2025. FinanceReasoning: Benchmarking financial numerical reasoning more credible, comprehensive and challenging. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15721–15749, Vienna, Austria. Association for Computational Linguistics.

Zhongyuan Wang, Richong Zhang, Zhijie Nie, and Hangyu Mao. 2025. General table question answering via answer-formula joint generation. *arXiv preprint arXiv:2503.12345*.

Zilong Ryan Wang, Hao Zhang, Chun-Liang Li, Julian M Eisenschlos, Vincent Perot, Zifeng Wang, Lesly Miculicich, Yasuhisa Fujii, Jingbo Shang, Chen-Yu Lee, and 1 others. 2024. Chain-of-table: Evolving tables in the reasoning chain for table understanding. In *International Conference on Learning Representations*, volume 2024, pages 55587–55610.

Ziwei Wang, Jiayuan Su, Mengyu Zhou, Huaxing Zeng, Mengni Jia, Xiao Lv, Haoyu Dong, Xiaojun Ma, Shi Han, and Dongmei Zhang. 2026. Sheetbrain: A neuro-symbolic agent for accurate reasoning over complex and large spreadsheets. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 33800–33808.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances*

in neural information processing systems, 35:24824–24837.

Haolun Wu, Zhenkun Li, and Lingyao Li. 2025. Can llm agents really debate? a controlled study of multi-agent debate in logical reasoning.

Yijia Xiao, Edward Sun, Di Luo, and Wei Wang. 2024. Tradingagents: Multi-agents llm financial trading framework. *arXiv preprint arXiv:2412.20138*.

Qianqian Xie, Weiguang Han, Zhengyu Chen, Ruoyu Xiang, Xiao Zhang, Yueru He, Mengxi Xiao, Dong Li, Yongfu Dai, Duanyu Feng, and 1 others. 2024. Finben: A holistic financial benchmark for large language models. *Advances in Neural Information Processing Systems*, 37:95716–95743.

Qianqian Xie, Weiguang Han, Xiao Zhang, Yanzhao Lai, Min Peng, Alejandro Lopez-Lira, and Jimin Huang. 2023. Pixiu: A large language model, instruction data and evaluation benchmark for finance.

Hang Yan, Xinyu Che, Fangzhi Xu, Qiushi Sun, Zichen Ding, Kanzhi Cheng, Jian Zhang, Tao Qin, Jun Liu, and Qika Lin. 2026. Tide: Trajectory-based diagnostic evaluation of test-time improvement in llm agents.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.

Yangyang Yu, Zhiyuan Yao, Haohang Li, Zhiyang Deng, Yuechen Jiang, Yupeng Cao, Zhi Chen, Jordan W Suchow, Zhenyu Cui, Rong Liu, and 1 others. 2024. Fincon: A synthesized llm multi-agent system with conceptual verbal reinforcement for enhanced financial decision making. *Advances in Neural Information Processing Systems*, 37:137010–137045.

Shaowei Zhang and Deyi Xiong. 2025. Debate4math: Multi-agent debate for fine-grained reasoning in math. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 16810–16824.

Tengxun Zhang, Hongfei Xu, Josef van Genabith, Deyi Xiong, and Hongying Zan. 2023. Napg: Non-autoregressive program generation for hybrid tabular-textual question answering. In *CCF International Conference on Natural Language Processing and Chinese Computing*, pages 591–603. Springer.

Xiaokang Zhang, Sijia Luo, Bohan Zhang, Zeyao Ma, Jing Zhang, Yang Li, Guanlin Li, Zijun Yao, Kangli Xu, Jinchang Zhou, and 1 others. 2025. Tablellm: Enabling tabular data manipulation by llms in real office usage scenarios. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 10315–10344.

Yunjia Zhang, Jordan Henkel, Avrilia Floratou, Joyce Cahoon, Shaleen Deep, and Jignesh M. Patel. 2024. Reactable: Enhancing react for table question answering. *Proc. VLDB Endow.*, 17(8):1981–1994.

Yilun Zhao, Yunxiang Li, Chenying Li, and Rui Zhang. 2022. Multihiertt: Numerical reasoning over multi hierarchical tabular and textual data. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6588–6600.

Yilun Zhao, Hongjun Liu, Yitao Long, Rui Zhang, Chen Zhao, and Arman Cohan. 2024a. Financemath: Knowledge-intensive math reasoning in finance domains. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12841–12858.

Yilun Zhao, Yitao Long, Hongjun Liu, Ryo Kamoi, Linyong Nan, Lyuhao Chen, Yixin Liu, Xiangru Tang, Rui Zhang, and Arman Cohan. 2024b. Docmath-eval: Evaluating math reasoning capabilities of llms in understanding long and specialized documents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16103–16120.

Fengbin Zhu, Wenqiang Lei, Youcheng Huang, Chao Wang, Shuo Zhang, Jiancheng Lv, Fuli Feng, and Tat-Seng Chua. 2021. Tat-qa: A question answering benchmark on a hybrid of tabular and textual content in finance. In *Proceedings of the 59th annual meeting of the Association for Computational Linguistics and the 11th international joint conference on natural language processing (volume 1: long papers)*, pages 3277–3287.

Appendices

This appendix provides supplementary material to support the main findings of this work. It is organized as follows:

- Appendix A describes the evaluation datasets, including task format, input modality, evaluation split, and metric.
- Appendix B reports the state-of-the-art reference tables used for comparison in the main experiments.
- Appendix C presents a qualitative example of MOCA-AGENT on FinQA, illustrating how claims are generated, traded, cleared, and used for program synthesis.
- Appendix D provides the prompt library used by the catalog builder, specialist trader agents, synthesizer, verifier, repair module, selector committee, and conflict arbiter.

Key Tables

- Table A.1: Summary of the *ten* evaluation benchmarks, including task type, input format, evaluation split, and metric.
- Table B.1: Overall performance of different LLMs on three financial reasoning datasets.
- Table B.2: Per-category accuracy on the FinanceMath validation split.
- Table B.3: State-of-the-art reference results for HiTab.
- Table B.4: State-of-the-art reference results for MultiHiertt.
- Table B.5: State-of-the-art reference results for TabMWP.
- Table B.7: FinChart-Bench leaderboard across True/False, multiple-choice, and open question-answering subtasks.
- Table B.8: ESGenius accuracy under the retrieval-augmented generation setting.
- Table B.9: Per-dataset internal diagnostics for MoCA-Agent, including execution reliability, code–answer consistency, and mean market confidence.
- Table C.1: Claim catalog for the FinQA *recourse debt* example.
- Table C.2: Cleared claim market for the FinQA *recourse debt* example.
- Table D.1: Decoding settings for each LLM call in the MoCA-Agent pipeline.

Key Figures

- Figure D.1: Prompt template for the claim-catalog builder.
- Figure D.2: Prompt template for the Extractor trader.
- Figure D.3: Prompt template for the Formula trader.
- Figure D.4: Prompt template for the Accountant trader.
- Figure D.5: Prompt template for the Skeptic trader.
- Figure D.6: Prompt template for the synthesizer.
- Figure D.7: Prompt template for market-aware repair.
- Figure D.8: Prompt template for the hybrid selector committee.
- Figure D.9: Prompt template for the conflict arbiter.

A Dataset Details

This appendix gives the per-dataset details summarized in §4.1. Table A.1 provides a one-glance overview; the paragraphs that follow elaborate on each dataset's source, scale, evaluation split, modality, and answer type.

FinQA (Chen et al., 2021). A widely used financial numerical-reasoning benchmark built from S&P 500 10-K filings. Each example pairs a short narrative passage with one or two tables and a numerical question whose gold answer is derived from a short expert-written Python program plus a list of gold supporting facts. The dataset contains 8,281 QA pairs split into train/dev/test (6,251/883/1,147); we evaluate on the 1,147-item public test split and report execution accuracy.

DocMath-Eval (Simplong, Complong) (Zhao et al., 2024b). A benchmark for math reasoning over *long* financial documents. DocMath-Eval has four subsets; we use the two long-context ones: *DM-Simplong*, re-annotated from MultiHiertt, requires simple numerical reasoning over a long document with multiple tables; and *DM-Complong*, annotated from scratch by the original authors, requires *complex* numerical reasoning over very long documents (average context near 18k words) with multiple tables and gold Python programs. We use the public `testmini` splits (100 for Simplong, 300 for Complong) so that our numbers are directly comparable to the SOTA table in (Zhao et al., 2024b).

FinanceMath (Zhao et al., 2024a). A college-level, knowledge-intensive finance math benchmark of 1,200 problems spanning seven finance subdomains (Quantitative Analysis, Asset Classes & Derivatives, Accounting, Risk Management, Portfolio Management, Market Analysis & Economics, and Corporate & Securities Issuance).

Cat.	Dataset	Source / Venue	Eval N	Split	Modality	Answer type
C1	FinQA	10-K filings; EMNLP 2021	1,147	test	text + table	numeric (program)
C1	DocMath-Simplong	long fin. docs; ACL 2024	100	testmini	text + multi-table	numeric (program)
C1	DocMath-Complong	long fin. docs; ACL 2024	300	testmini	text + multi-table	numeric (program)
C1	FinanceMath	finance textbooks; ACL 2024	200	validation	text + table	numeric (program)
C2	HiTab	stats reports + Wikipedia; ACL 2022	1,584	test	hierarchical table	numeric / span
C2	MultiHiertt	financial reports; ACL 2022	1,044	dev	text + multi-hier. table	numeric (program)
C2	TabMWP	grade-school MWP; ICLR 2023	1,000	test (sub)	semi-structured table	numeric / MC
C2	WikiTableQuestions	Wikipedia tables; ACL 2015	4,344	test	semi-structured table	span / numeric
C3	ESGenius	ESG standards; EMNLP 2025	1,136	full	text (RAG passage)	4-way MCQ
C4	FinChart-Bench (TF)	real-world fin. charts; 2025	2,384	full	chart image	True/False
C4	FinChart-Bench (MC)	real-world fin. charts; 2025	2,350	full	chart image	4-way MC
C4	FinChart-Bench (QA)	real-world fin. charts; 2025	2,284	full	chart image	open numeric

Table A.1: Summary of the ten benchmarks (twelve evaluation tracks) used in this paper. "Eval N" is the number of evaluation items in our run; "Split" names the dataset's official split. Answer type denotes the dominant output format: *numeric (program)* benchmarks pair gold answers with executable Python; *span* benchmarks expect a denotation from the table; *MC* / MCQ are multiple-choice.

Each problem combines textual and tabular content and is paired with an expert-written Python solution; estimated human-expert accuracy is 92%. Following the original release, we evaluate on the 200-sample validation split.

HiTab (Cheng et al., 2022). A cross-domain QA / NLG benchmark for *hierarchical* tables sourced from statistical reports and Wikipedia. HiTab contains 10,686 QA pairs and descriptive sentences over 3,597 tables, with fine-grained entity and quantity-alignment annotations. We evaluate on the 1,584-item public test split, the standard split for the QA task.

MultiHiertt (Zhao et al., 2022). A numerical-reasoning benchmark over hybrid hierarchical-tabular and textual data from real financial reports: each document contains multiple hierarchical tables plus surrounding text, and the gold answer requires multi-step symbolic reasoning across them. Following prior work, we evaluate on the public dev split of 1,044 items (the test set's answers are private).

TabMWP (Lu et al., 2022). A tabular math-word-problem benchmark of 38,431 grade-school problems, each accompanied by a tabular context (rendered as image, semi-structured text, and structured table). About 74.7% of questions are free-text numerical and 25.3% are multiple-choice with text answers; the official 6:2:2 split yields a 23,059/7,686/7,686 train/dev/test partition. To keep cost comparable to the other benchmarks we evaluate on a fixed 1,000 random subset of the test split (seed fixed in our public code).

WikiTableQuestions (WTQ) (Pasupat and Liang, 2015). A widely used open-domain table-QA benchmark of 22,033 complex questions over 2,108 semi-structured Wikipedia tables, requiring operations such as comparison, aggregation, and arithmetic. We evaluate on the official 4,344-item test split with denotation accuracy.

ESGenius (He et al., 2025). A benchmark of 1,136 expert-validated multiple-choice questions covering Environmental, Social, and Governance (ESG) and sustainability topics. Each question is linked to its source passage in a curated corpus of 231 authoritative documents (GRI, SASB, TCFD, IFRS/ISSB, CDP, IPCC, and UN SDG sources) and supports both zero-shot and Retrieval-Augmented Generation (RAG) protocols. We pass each question's gold source passage as the context to MoCA-Agent, which is the oracle-RAG-equivalent setting and directly comparable to the open-source RAG column of the original leaderboard.

FinChart-Bench (Shu et al., 2025). The first benchmark specifically targeting real-world financial charts: 1,200 chart images collected from 2015 to 2024, each annotated with three question types: True/False (TF), Multiple-Choice (MC), and open Question-Answering (QA), totalling 7,016 items. We evaluate MoCA-Agent on all three subtasks (TF: 2,384; MC: 2,350; QA: 2,284 items in our run). The QA subtask requires reading numbers off the chart and answering numerical queries, exercising both the multimodal extractor and the operation-

aware verifier; it is also the subtask where the published gap between the best proprietary and best open-source VLM is largest, which is why §4 focuses on it.

B State-of-the-Art Reference Tables

This appendix collects the full per-method SOTA tables for every dataset evaluated in the main paper. Each table is reproduced from the cited reference and a final row is appended showing MoCA-Agent under the same protocol; per-category scores not annotated by the dataset's public release at evaluation time are marked "–".

B.1 FinQA / DocMath-Simplong / DocMath-Complong

Table B.1 reproduces the unified cross-dataset comparison from Zhao et al. (2024b), the most recent finance-numerical-reasoning leaderboard that runs the same models on all three datasets under a single protocol. The original table groups **30+ baselines** into four bands: *(i)* proprietary frontier LLMs (GPT-4o, GPT-o1-preview, GPT-o3-mini, GPT-4.5, DeepSeek-V3 and DeepSeek-R1); *(ii)* the largest open-source models in the 70B to 405B range (Llama-3/3.1/3.3, Llama-4-Scout, Qwen2.5-72B and its math-tuned variant, the DeepSeek-R1-Distill family); *(iii)* the 30B class (Qwen2.5-32B, Qwen3-32B, QwQ-32B, DeepSeek-R1-Distill-Qwen-32B, Limo, S1-32B); *(iv)* the 7B to 14B class, including domain-specialised reasoners (FinR1-7B, Dianjin-R1-7B); and *(v)* the two strongest finance-tuned baselines, Fino1-8B and Fino1-14B.

The most striking pattern in this table is how badly even very strong backbones collapse as the context lengthens and the reasoning becomes multi-hierarchical. On FinQA, which has short context and a single or paired table, Qwen2.5-72B-Instruct already reaches 73.38%, and Fino1-14B sets the prior public ceiling at 74.18%. On DM-Simplong, the same models drop into the 50 to 59 range, with GPT-4o the strongest prior baseline at 60.0%. On DM-Complong the gap widens further: Qwen2.5-72B-Instruct collapses from 73.38 (FinQA) to 14.67 (Complong), Qwen2.5-Math-72B collapses to 5.0, Llama-4-Scout to 0.67, and even the strongest proprietary system in the table (DeepSeek-V3) only reaches 42.33. MoCA-Agent at the relatively modest 27B parameter scale reaches **78.29 / 70.00 / 50.67** on FinQA / DM-Simplong / DM-Complong, posting the best score in *every* column and an average of **66.32**, roughly +9 points over the strongest prior cross-row average (GPT-4o at 57.27). Crucially, the gain widens with task difficulty (+4.1 / +10.0 / +8.3 against the strongest prior on each column), which is consistent with the silent-error-class diagnosis in §3: the longer and more hierarchical the input, the more chances there are for a single misread cell to corrupt the final number, and the more leverage the trader market gets out of having each cell scrutinised by four different roles.

B.2 FinanceMath Per-Category

Table B.2 reproduces the headline leaderboard from Zhao et al. (2024a) (their Table 2, dev set, 200 samples), which evaluates each model under both *Program-of-Thought* (PoT) and *Chain-of-Thought* (CoT) prompting and reports per-category accuracy across seven finance subdomains: *Quantitative Analysis & Valuation*, *Asset Classes & Derivatives*, *Accounting*, *Risk Management*, *Portfolio Management & Strategy*, *Market Analysis & Economics*, and *Corporate & Securities Issuance*. The original paper benchmarks **44 models**; we keep the 10 proprietary baselines and the 10 strongest open-source baselines from the original release for space, and refer to Zhao et al. (2024a) for the full list.

Several patterns from the original leaderboard help frame our result. First, no proprietary model exceeds 67% average accuracy in either prompting mode; the strongest baseline overall is GPT-4o (PoT, 67.0%), with Claude-3.5-Sonnet a close second on the CoT side (60.6%). Second, the open-source side is dominated by very large backbones (DeepSeek-Coder-V2 236B, DeepSeek-V2 236B, Llama-3.1 405B, Mistral-Large 123B); even at 405B parameters, Llama-3.1 only matches the mid-range proprietary models. Third, performance is highly uneven across subdomains: *Risk Management* (small, formula-rich) and *Quantitative Analysis* (computation-heavy) tend to be the hardest categories for the closed-form prompting baselines, while *Portfolio Management* and *Market Analysis* are comparatively easier.

Because MoCA-Agent is a code agent that always emits an executable Python solution, we report it in the PoT column. MoCA-Agent reaches **76.0%** overall (+9.0 over the strongest PoT baseline, GPT-4o at 67.0%, and +15.1 over the strongest CoT baseline, Claude-3.5-Sonnet at 60.6%), with the per-category breakdown com-

Models	FinQA	DM-Simplong	DM-Complong	Average
GPT-4o	72.49	60.00	39.33	57.27
GPT-o1-preview	49.07	56.00	36.67	47.25
GPT-o3-mini	60.87	59.00	35.00	51.62
DeepSeek-V3	73.20	53.00	42.33	56.18
DeepSeek-R1	65.13	53.00	38.67	52.27
GPT-4.5	68.94	59.00	39.33	55.76
Llama-4-Scout	70.45	52.00	0.67	41.04
Llama-3-70B-Instruct	58.92	41.00	13.67	37.86
Llama-3.1-70B-Instruct	63.18	48.00	34.33	48.50
Llama-3.3-70B-Instruct	68.15	54.00	32.00	51.38
Qwen2.5-72B-Instruct	73.38	59.00	14.67	49.02
Qwen2.5-Math-72B-Instruct	69.74	42.00	5.00	38.91
DeepSeek-R1-Distill-Llama-70B	66.73	53.00	30.67	50.13
Qwen2.5-32B-Instruct	73.11	56.00	30.00	53.04
Qwen3-32B	64.15	51.00	26.00	47.05
QwQ-32B	61.22	46.00	20.00	42.41
DeepSeek-R1-Distill-Qwen-32B	65.48	55.00	24.67	48.38
Limo	63.44	45.00	15.33	41.26
S1-32B	66.81	53.00	24.00	47.94
Qwen2.5-14B-Instruct	67.44	59.00	26.67	51.04
Qwen3-14B	64.33	49.00	24.00	45.78
DeepSeek-R1-Distill-Qwen-14B	63.27	44.00	21.00	42.76
DeepSeek-R1-Distill-Llama-8B	45.96	33.00	15.67	31.54
Llama-3-8B-Instruct	41.97	29.00	6.00	25.66
Llama-3.1-8B-Instruct	54.13	34.00	14.30	34.14
Qwen2.5-7B-Instruct	55.37	41.00	17.67	38.01
Qwen3-8B	62.11	46.00	17.67	41.93
FinR1-7B	58.74	37.00	13.67	36.47
Dianjin-R1-7B	60.20	35.00	14.67	36.62
Fino1-8B	73.03	56.00	26.33	51.79
Fino1-14B	74.18	55.00	27.33	52.17
MoCA-Agent (ours)	**78.29**	**70.00**	**50.67**	**66.32**

Table B.1: Overall performance of different LLMs on three financial reasoning datasets. Non-MoCA-Agent rows reproduced from Zhao et al. (2024b).

puted by joining our run's predictions with the validation set's `topic` field. The gains are largest where the original paper reports the lowest baselines: **91.67%** on Quantitative Analysis (vs. GPT-4o PoT 75.0%), **74.60%** on Derivatives, and **74.36%** on Accounting. The lowest MoCA-Agent cell is on Risk Management (**55.56%**), but the corresponding sub-sample has only n=9 items and the variance is correspondingly wide; the per-sample n's are listed in the table caption so readers can compute confidence intervals on the fly.

B.3 HiTab

Table B.3 collects the strongest results published on HiTab (Cheng et al., 2022) since the dataset's release, split into two methodological bands. The *fine-tuning* band is the larger of the two and includes both encoder-style baselines (TAPEX-Large (Liu et al., 2021), 45.6%) and a sequence of table-specialised decoder models trained at 7B parameters (TableLlama 60.5%, TableLLM (Zhang et al., 2025) 43.9%, $\text{TAMO}^{+}_{\text{SFT}}$ 63.9%, TableGPT2 (Su et al., 2024) 70.3%); the strongest open-source baseline in this band is TabAF (Wang et al., 2025) at 78.41% on a Qwen2.5-Coder-7B backbone. The *prompting / in-context learning* band is led by SS-CoT (Shi et al., 2025) on Llama-3.1-70B at 79.10%, followed by GraphOTTER variants (73.74% with Qwen2-72B-Instruct, 67.76% with Gemini-1.5-Flash) and API-Assisted code prompting ((Cao and Liu, 2025), 69.30%).

MoCA-Agent at 27B parameters reaches 77.27%, which places it within -1.83 points of the absolute leader (SS-CoT on a 70B backbone) and within -1.14 points of the strongest fine-tuned coder (TabAF on a 7B backbone). HiTab is the one benchmark in the paper where MoCA-Agent

Model	Size	Notes	Quantitative		Derivatives		Accounting		Management		Portfolio		Economics		Corporate		Avg.	
			PoT	CoT	PoT	CoT	PoT	CoT	PoT	CoT	PoT	CoT	PoT	CoT	PoT	CoT	PoT	**CoT**
Proprietary LLMs																		
GPT-4o			75.0	45.8	58.8	55.4	60.3	69.4	82.9	66.3	77.8	69.4	62.5	58.9	67.0	56.9	67.0	60.9
Claude-3.5-Sonnet			73.6	55.6	54.1	51.8	66.7	69.9	75.6	63.9	72.2	72.2	64.3	58.9	62.4	60.6	64.8	60.6
Claude-3-Opus			66.7	56.9	53.5	45.2	62.1	59.8	79.5	64.9	72.2	83.3	51.8	46.4	59.6	45.0	62.9	54.7
GPT-4-Turbo			59.7	38.9	49.8	42.2	50.7	64.8	72.2	56.6	61.1	50.0	57.1	44.6	50.5	47.7	56.2	50.9
Gemini-1.5-Pro			68.1	50.0	53.1	30.7	56.6	55.2	69.8	57.6	58.3	63.9	51.8	55.4	50.5	44.0	58.2	47.0
GPT-4o-Mini			65.3	36.1	46.9	29.7	48.4	47.5	69.3	46.8	50.0	38.9	57.1	41.1	51.4	45.9	54.3	40.3
Gemini-1.5-Flash			69.4	33.3	43.6	28.7	52.0	48.9	67.8	49.8	58.3	61.1	50.0	37.5	47.7	34.9	53.6	40.1
Claude-3-Sonnet			59.7	37.5	37.0	28.4	48.0	43.4	66.8	48.8	47.2	55.6	48.2	33.9	48.6	35.8	49.4	38.6
Claude-3-Haiku			34.7	31.9	19.8	26.4	33.8	43.4	44.9	41.5	41.7	44.4	25.0	33.9	36.7	30.3	32.0	35.1
GPT-3.5-Turbo			47.2	25.0	24.4	16.5	29.2	29.2	51.2	33.2	27.8	22.2	37.5	21.4	32.1	23.8	34.3	24.6
Open-source LLMs (top 10 of 35)																		
DeepSeek-Coder-V2	236B	Code	38.5	41.0	62.5	66.7	47.6	46.0	75.0	61.1	33.3	44.4	79.0	52.6	60.0	50.0	55.5	51.0
DeepSeek-V2	236B	MoE	38.5	46.2	66.7	79.2	47.6	44.4	75.0	52.8	44.4	33.3	73.7	47.4	30.0	40.0	54.5	50.0
Llama-3.1	405B		41.0	51.3	54.2	54.2	46.0	33.3	80.6	47.2	33.3	11.1	68.4	31.6	40.0	50.0	53.5	41.5
Mistral-Large	123B		38.5	51.3	50.0	45.8	38.1	30.2	77.8	36.1	44.4	22.2	73.7	31.6	40.0	10.0	50.5	36.0
Llama-3.1	70B		25.6	38.5	41.7	50.0	33.3	19.0	66.7	47.2	55.6	22.2	68.4	47.4	30.0	30.0	43.0	35.0
Qwen2	72B		23.1	33.3	29.2	45.8	23.8	22.2	41.7	47.2	22.2	22.2	68.4	31.6	10.0	30.0	31.0	33.0
Llama-3	70B		33.3	38.5	37.5	45.8	33.3	22.2	66.7	33.3	33.3	22.2	68.4	31.6	30.0	30.0	43.0	31.5
Phi-3-Medium	14B		20.5	28.2	37.5	50.0	25.4	19.0	55.6	41.7	22.2	22.2	52.6	42.1	40.0	20.0	34.5	31.0
DeepSeek-Coder-V2-Lite	16B	Code	23.1	28.2	25.0	29.2	27.0	23.8	50.0	33.3	11.1	22.2	42.1	31.6	10.0	20.0	30.0	27.5
Mixtral-8x22B	141B	MoE	7.7	28.2	4.2	45.8	0.0	19.0	25.0	30.6	11.1	11.1	21.0	26.3	10.0	20.0	9.5	26.5
MoCA-Agent (ours)	27B	Agent	–	**91.67**	–	**74.60**	–	**74.36**	–	**55.56**	–	**68.42**	–	**75.00**	–	**70.00**	–	**76.00**

Table B.2: Per-category accuracy on the FinanceMath validation split. Non-MoCA-Agent rows reproduced from Zhao et al. (2024a), Table 2 (top 10 of 35 open-source rows shown for space; the full 44-model list is in the original paper). MoCA-Agent per-category numbers were computed by joining our run's predictions with the validation set's `topic` field (Quantitative=Quantitative Analysis & Valuation, n=36; Derivatives=Asset Classes & Derivatives, n=63; Accounting, n=39; Management=Risk Management, n=9; Portfolio=Portfolio Management & Strategy, n=19; Economics=Market Analysis & Economics, n=24; Corporate=Corporate & Securities Issuance, n=10). Because MoCA-Agent is a code agent, accuracy is reported in the PoT column; the CoT column is left empty.

does *not* set a new SOTA. Two structural reasons explain the gap. First, the top entries in both methodology bands were directly trained or aggressively in-context-engineered on hierarchical-table column-header normalisation, a pre-processing step that is largely orthogonal to the market-of-claims mechanism itself; an out-of-the-box agent with a generic catalog builder does not gain from this signal. Second, HiTab questions are typically single-cell or single-aggregate look-ups over a clean hierarchical header, so the cell-extraction adversary that drives MoCA-Agent's gains on MultiHiertt and DocMath is comparatively muted here. We expect that a HiTab-specific catalog template would close this gap, but we deliberately do not adapt prompts per dataset in this work.

Method	Backbone	HiTab
Fine-tuning approaches		
TAPEX-Large (Liu et al., 2021)	BART-Large	45.60
TableLlama	Llama-2-7B	60.48
TableLLM (Zhang et al., 2025)	Qwen2-7B	43.88
$\text{TAMO}^{+}_{\text{SFT}}$	Llama-2-7B	63.89
TableGPT2 (Su et al., 2024)	Qwen2.5-7B	70.27
TabAF (Wang et al., 2025)	Qwen2.5-Coder-7B	78.41
Prompting / in-context learning with LLMs		
API-Assisted (Cao and Liu, 2025)	Codex	69.30
GraphOTTER	Qwen2-72B-Instruct	73.74
GraphOTTER	Gemini-1.5-Flash	67.76
SS-CoT (Shi et al., 2025)	Llama-3.1-70B	79.10
MoCA-Agent (ours)	Qwen3.6-27B	77.27

Table B.3: State-of-the-art methods on HiTab (Cheng et al., 2022) published after the original benchmark paper. All numbers are exact-match / execution accuracy (%) on the HiTab test split.

B.4 MultiHiertt

Table B.4 reproduces the most up-to-date MultiHiertt (Zhao et al., 2022) leaderboard, organised into four methodological bands by the recent Fortune paper (Cao et al., 2025) (which is itself the strongest non-MoCA-Agent system in the table). The *fine-tuning* band contains the original benchmark baseline NAPG (Zhang et al., 2023) on RoBERTa-large (44.19%). The *zero-shot textual* band is led by o1 (38.31%) with GPT-4o at 36.11% and GPT-4o-mini at 22.41%; the matched *zero-shot symbolic / formula* band lifts every entry by 6 to 11 points (o1 climbs to 48.95%, GPT-4o to 38.41%), confirming that writing an explicit formula is the dominant lever on this dataset. The strongest pub-

lished prior result comes from the *reinforcement-learning* band: Fortune's `RL w/ CS (Formula)` variant on Qwen2.5-Coder-7B reaches 56.78%, with Fortune++ (Cao et al., 2025) at 51.73% and the matched `RL w/ CS (Text)` configurations in the 49 to 55 range.

MoCA-Agent at 27B parameters reaches **71.17%**, a +14.39-point gain over the strongest RL-trained system on a Qwen2.5-Coder-7B backbone. This is the single largest published gap in the paper and the most direct evidence for the market-of-claims hypothesis. MultiHiertt is precisely the regime that exercises the failure mode the market is designed to suppress: joining several hierarchical tables, each with its own header structure, into a single multi-step formula, and the gap to the strongest formula-reasoning Frontier model (o1 at 48.95%) is even larger (+22.22). Note also that MoCA-Agent exceeds the strongest RL system without any task-specific training: the same prompts and the same backbone are used on all ten benchmarks.

Method	Backbone	MultiHiertt
Fine-tuning approaches		
NAPG (Zhang et al., 2023)	RoBERTa-large	44.19
Prompting / zero-shot LLMs (textual reasoning)		
GPT-4o-mini (Cao et al., 2025)	GPT-4o-mini	22.41
GPT-4o (Cao et al., 2025)	GPT-4o	36.11
o1 (Cao et al., 2025)	o1	38.31
Prompting / zero-shot LLMs (symbolic / formula reasoning)		
GPT-4o-mini (Cao et al., 2025)	GPT-4o-mini	28.26
GPT-4o (Cao et al., 2025)	GPT-4o	38.41
o1 (Cao et al., 2025)	o1	48.95
Reinforcement-learning-based approaches		
Fortune (Cao et al., 2025)	Qwen2.5-Coder-7B	40.85
RL w/ CS (Text) (Cao et al., 2025)	Llama-3.1-8B	49.34
RL w/ CS (Formula) (Cao et al., 2025)	Llama-3.1-8B	54.55
RL w/ CS (Text) (Cao et al., 2025)	Qwen2.5-Coder-7B	54.25
RL w/ CS (Formula) (Cao et al., 2025)	Qwen2.5-Coder-7B	56.78
Fortune++ (Cao et al., 2025)	Qwen2.5-Coder-7B	51.73
MoCA-Agent (ours)	Qwen3.6-27B	**71.17**

Table B.4: State-of-the-art methods on MultiHiertt (Zhao et al., 2022) published after the original benchmark paper. All numbers are exact-match / execution accuracy (%) on the MultiHiertt dev split.

B.5 TabMWP

Table B.5 collects the published state of the art on TabMWP (Lu et al., 2022), organised into three methodological bands: *fine-tuning* (TaCo variants up to 92.91% on TAPEX-large; the ToRA / ToRA-Code family up to 74.0% on Llama-2-70B); *prompting / chain-of-thought* (PoT-SC on Codex 81.80%, CoT and PoT on ChatGPT in the 82 to 89 range, CoT on GPT-4 90.81%); and *tool-augmented / agent-based methods* which dominate the upper half of the leaderboard. Within the agent band, CRITIC (Gou et al., 2024) reaches 89.0% on ChatGPT, CRAFT 88.4%, CoS-Planning 90.0%, PoT+Doc (Hsieh et al., 2023) 92.69%, and Chameleon (Lu et al., 2023) 93.28%; the strongest prior result is CREATOR (Qian et al., 2023) on ChatGPT at 94.70%.

MoCA-Agent at 27B reaches **96.00%**, a +1.30-point gain over CREATOR despite using a substantially smaller backbone than ChatGPT. The margin over the strongest non-agent prompting baseline (CoT on GPT-4 at 90.81%) is +5.19. TabMWP is comparatively "easy" for the top of the leaderboard, so the gain here is small in absolute terms but consistent with the rest of the paper: the trader market kills exactly the small population of silent mis-extraction errors that the strongest tool-augmented agents (which already operate close to ceiling) still produce.

Method	Backbone	TabMWP
Fine-tuning approaches		
TaCo	TAPEX-base	86.12
TaCo	TAPEX-large	92.91
ToRA	Llama-2-7B	42.40
ToRA	Llama-2-13B	47.20
ToRA	Llama-2-70B	74.00
ToRA-Code	CodeLlama-34B	70.50
Prompting / chain-of-thought with LLMs		
PoT-SC	Codex	81.80
CoT (Lu et al., 2023)	ChatGPT	82.03
RetICL	Codex	88.51
PoT (Lu et al., 2023)	ChatGPT	89.49
CoT (Lu et al., 2023)	GPT-4	90.81
Tool-augmented / agent-based methods		
CRITIC (Gou et al., 2024)	GPT-3	87.60
CRITIC (Gou et al., 2024)	ChatGPT	89.00
CRITIC (Gou et al., 2024)	Llama-2-70B	75.00
CRAFT	ChatGPT	88.40
CoS-Planning	ChatGPT	90.00
PoT+Doc (Hsieh et al., 2023)	ChatGPT	92.69
Chameleon (Lu et al., 2023)	ChatGPT	93.28
CREATOR (Qian et al., 2023)	ChatGPT	94.70
MoCA-Agent (ours)	Qwen3.6-27B	**96.00**

Table B.5: State-of-the-art methods on TabMWP (Lu et al., 2022) published after the original benchmark paper. All numbers are accuracy (%) on the TabMWP test split.

B.6 WikiTableQuestions

Table B.6 collects the published state of the art on WikiTableQuestions (WTQ) (Pasupat and Liang, 2015). The *fine-tuning* band starts at TAPAS-

large (BERT-large, 48.70%) and climbs through a sequence of table-specialised encoder-decoder models (TAPEX-Large (Liu et al., 2021) 59.10%, OmniTab (Jiang et al., 2022) 62.80%, TableLLM 53.59%, TableGPT2 (Su et al., 2024) 61.42%); the strongest fine-tuned baseline is TabAF (Wang et al., 2025) at 74.72% on Qwen2.5-Coder-7B. The *prompting / in-context learning* band is considerably more crowded, including Binder, Dater, LEVER, ReAcTable, Chain-of-Table (Wang et al., 2024), CABINET, Mix-SC, Norm-DP&Agent, SynTQA, and TIDE-Agent, all in the 64 to 75 range. The strongest non-agent prompting result is SS-CoT (Shi et al., 2025) on Llama-3.1-70B at 76.80%, and the strongest agent-style result is ARTEMIS-DA (Hussain, 2024) on GPT-4 at 80.80%, with TableMaster (Cao and Liu, 2025) on GPT-4o-mini close behind at 78.13%.

MoCA-Agent at 27B reaches **81.40%**, +0.60 over ARTEMIS-DA on GPT-4 and +2.97 over TableMaster on GPT-4o-mini, while using a substantially smaller open-source backbone. WTQ is comparatively "noisier" than the financial benchmarks: many gold answers are short denotations, and most errors are not silent miscomputations but ambiguous question-cell alignments, so the margin here is smaller than on MultiHiertt or DM-Complong. The fact that MoCA-Agent still moves the needle on this dataset suggests that even on general-domain open tables, the per-claim trading mechanism captures information that single-pass agents (and the strongest prompted GPT-4 systems) miss.

Method	Backbone	WTQ
Fine-tuning approaches		
TAPAS-large	BERT-large	48.70
TAPEX-Large (Liu et al., 2021)	BART-Large	59.10
OmniTab (Jiang et al., 2022)	BART-Large	62.80
TableLlama	Llama-2-7B	32.14
TableLLM (Zhang et al., 2025)	Qwen2-7B	53.59
TableGPT2 (Su et al., 2024)	Qwen2.5-7B	61.42
TabAF (Wang et al., 2025)	Qwen2.5-Coder-7B	74.72
Prompting / in-context learning with LLMs		
Binder	Codex	64.60
Dater	Codex	65.90
LEVER	Codex	65.80
ReAcTable	Codex	68.00
Chain-of-Table (Wang et al., 2024)	PaLM-2	67.31
CABINET	FLAN-T5/LLM	69.10
Mix-SC	GPT-3.5	73.60
Norm-DP&Agent	GPT-3.5	73.65
SynTQA	GPT-4	74.40
TIDE-Agent	GPT-3.5	75.00
SS-CoT (Shi et al., 2025)	Llama-3.1-70B	76.80
TableMaster (Cao and Liu, 2025)	GPT-4o-mini	78.13
ARTEMIS-DA (Hussain, 2024)	GPT-4	80.80
MoCA-Agent (ours)	Qwen3.6-27B	**81.40**

Table B.6: State-of-the-art methods on WikiTableQuestions (Pasupat and Liang, 2015) published after the original benchmark paper. All numbers are exact-match / denotation accuracy (%) on the WTQ test set.

B.7 FinChart-Bench

Table B.7 reproduces the full FinChart-Bench leaderboard from Shu et al. (2025) (their Table 3), which benchmarks 25 modern vision-language models across the three subtasks of the dataset: True/False (TF), Multiple-Choice (MC), and open Question-Answering (QA). The original table groups baselines into three columns. The *open-source* column spans general VLMs from 0.2B (nanoVLM) to 24B (Mistral 3.1); among these, Qwen2.5 VL 7B leads on TF (95.09%), Mistral 3.1 24B leads on MC (84.17%) and average (74.37%), and LLaMa 4 17B leads on QA (59.78%). The *chart fine-tuned* column shows that chart-specialised fine-tuning of small VLMs (UniChart, Matcha, ChartGemma, ChartInstruct variants) is essentially non-competitive on this benchmark: every entry in that column scores below 1% on QA and below 3% on MC, suggesting that chart-fine-tuning datasets do not transfer to the long-tail of real-world financial chart types. The *closed-source* column is led by Claude Sonnet 4 on average (84.32%) and on QA (63.59%), with o3 leading on TF (97.86%) and MC (92.17%); GPT-4.1, GPT-4.1-mini, o4-mini, and Gemini 2.5 Pro all score within a 4-point band on average accuracy.

MoCA-Agent runs the same Qwen3.6-27B trader market used on the text-only benchmarks, with only the table slot T replaced by a single VLM-generated transcription of the chart (see §3.6). On TF, MoCA-Agent scores 97.27%, tying Gemini 2.5 Pro and trailing o3 / o4-mini by less than one point. On MC, MoCA-Agent scores 84.85%, which leads every open-source entry (the next-best is Mistral 3.1 24B at 84.17%) and beats GPT-4o (77.57%) and GPT-4.1 (83.19%) on the closed-source side. The headline lift is on QA: MoCA-Agent reaches **74.65%**, +11.06 over Claude Sonnet 4 (63.59%), +11.19 over Gemini 2.5 Pro (63.46%), and +12.11 over GPT-4.1 (62.54%). The three-subtask average is **85.59%**, +1.27 over the strongest prior closed-source baseline (Claude Sonnet 4) and +11.22 over the strongest open-source baseline (Mistral 3.1 24B). The QA gain matters because that is the only subtask of the three that requires *open numerical* answers rather than

multiple-choice classification, exactly the regime in which a fluent but silently mis-extracted chart value can pass a single-pass VLM but is shorted by the ACCOUNTANT and SKEPTIC roles in the market.

Model	TF	MC	QA	Avg.
Open Source				
LLaMa 3.2 (11B)	63.46	64.00	5.12	44.65
LLaVa v1.6 (7B)	63.30	36.94	1.84	34.47
Qwen2.5 VL (7B)	95.09	82.81	37.29	72.16
Qwen2 VL (7B)	91.86	19.40	24.77	45.76
DeepSeek VL2 (2.8B)	57.97	6.77	4.73	23.50
Gemma 3 (12B)	89.14	70.94	27.22	62.89
Mistral 3.1 (24B)	92.95	84.17	44.90	74.37
Sa2VA (8B)	86.62	75.79	19.43	61.12
nanoVLM (222M)	53.90	2.43	0.22	19.19
Cosmos (7B)	70.93	43.49	20.18	45.22
MedGemma (4B)	53.27	46.30	3.72	34.81
Blip 2 (2.7B)	49.79	0.77	0.00	17.17
LLaMa 4 (17B)	89.56	59.70	59.78	69.89
Chart Fine-tuned				
UniChart (201M)	51.05	0.00	0.57	17.52
Matcha (282M)	49.54	0.00	0.70	17.05
ChartGemma (3B)	48.70	0.29	0.70	16.87
ChartInstruct-llama (7B)	51.09	0.17	1.97	18.05
ChartInstruct-T5 (3B)	62.12	2.30	1.71	22.43
Closed Source				
GPT 4o	93.92	77.57	57.59	76.62
GPT 4.1 mini	91.61	86.77	60.30	79.80
GPT 4.1	96.18	83.19	62.54	80.88
o4 mini	97.61	91.96	60.18	83.53
Gemini 2.5 Pro	97.27	89.70	63.46	83.73
o3	97.86	92.17	60.79	83.89
Claude Sonnet 4	96.94	91.66	63.59	84.32
MOCA-AGENT (ours)	97.27	84.85	**74.65**	**85.59**

Table B.7: FinChart-Bench leaderboard reproduced from Shu et al. (2025), Table 3, with MOCA-AGENT (Qwen3.6-27B + VLM transcription) evaluated on all three subtasks (TF: 2,384, MC: 2,350, QA: 2,284 in our run). **Bold**: new overall best across the three baseline groups. MOCA-AGENT sets a new state-of-the-art on QA and on the three-subtask average, leads every open-source and chart-fine-tuned entry on MC, and ties Gemini 2.5 Pro on TF.

B.8 ESGenius

Table B.8 reproduces the full ESGenius leaderboard from He et al. (2025) under the **RAG** setting, spanning **37 open-source models** from four families (DeepSeek, Google Gemma, Meta Llama, Alibaba Qwen) at sizes ranging from 0.5B to 70B parameters, and covering both base and instruction-tuned ("I-T") variants. We drop the proprietary API rows (GPT-4o, GPT-4o-mini, o3, o4-mini, DeepSeek-R1, DeepSeek-V3, Qwen2.5-Max) because the original paper does not report their RAG numbers; the zero-shot baselines for all 50 models are summarised in Table 3 of the main paper.

Three patterns from the original leaderboard are worth keeping in mind when reading the table. *(i) Scale matters.* Within each family, accuracy climbs roughly monotonically with parameter count: Gemma-3 1B reaches 0.5977 with instruction-tuning, climbing to 0.8380 at 12B and 0.8336 at 27B; Qwen2.5 climbs from 0.5396 at 0.5B to 0.8257 at 72B with instruction-tuning. *(ii) Instruction-tuning helps substantially.* Across nearly every base model where both variants are reported, the instruction-tuned variant adds 3 to 35 points of RAG accuracy (e.g., Gemma-3 27B jumps from 0.5229 to 0.8336 with IT). *(iii) Reasoning-focused models are competitive but not dominant.* The DeepSeek-R1-Distill family ("Rea" = Yes) tops out at 0.8170 (Llama-70B) and 0.8143 (Qwen-32B), which is comparable to but not above the best non-reasoning RAG entries. The strongest prior open-source RAG configuration in the table is Gemma-3 12B with IT at 0.8380.

MOCA-AGENT reaches **0.8688** at 27B, a $+0.0308$-point gain over the strongest open-source RAG entry, despite using no static retriever of its own. MOCA-AGENT's loader simply passes each question's released supporting passage into the agent's context, making the setting oracle-RAG-equivalent and directly comparable to the open-source RAG column. The gap to the strongest zero-shot model in the companion table is even larger ($+14.34$ over o3 at 0.7254). Two factors drive the lift: the SKEPTIC role frequently shorts plausible-but-wrong distractor options that exploit lexical overlap with the passage, which the ESGenius paper itself identifies as the dominant failure mode on this dataset, and the catalog builder forces the model to commit each option to a typed factual sub-claim before scoring, eliminating the answer-flipping fluctuations that hurt larger LLMs in the zero-shot column.

B.9 Internal Diagnostics

Table B.9 summarizes MOCA-AGENT's internal behaviour. Three observations are worth flagging. First, the code execution rate is $\geq 96\%$ on every benchmark and hits 100% on TabMWP and ESGenius, confirming that the synthesizer's claim-grounded prompt almost never produces unrunnable code. Second, *code-answer consistency tracks the execution rate* on every dataset, which means the answer-extraction failure mode that

Family	Model	Size	IT	Rea	RAG
DeepSeek	R1-Distill-Qwen	1.5B		✓	0.4305
	R1-Distill-Qwen	7B		✓	0.6505
	R1-Distill-Qwen	14B		✓	0.8046
	R1-Distill-Qwen	32B		✓	0.8143
	R1-Distill-Llama	8B		✓	0.6250
	R1-Distill-Llama	70B		✓	0.8170
Gemma	Gemma-3	1B			0.2526
	Gemma-3	1B	✓		0.5977
	Gemma-3	4B			0.6860
	Gemma-3	4B	✓		0.7518
	Gemma-3	12B			0.6857
	Gemma-3	12B	✓		0.8380
	Gemma-3	27B			0.5229
	Gemma-3	27B	✓		0.8336
Llama	Llama-3	8B			0.7324
	Llama-3.1	8B			0.7650
	Llama-3.1	8B	✓		0.7993
	Llama-3.2	1B			0.3680
	Llama-3.2	1B	✓		0.6452
	Llama-3.2	3B			0.6831
	Llama-3.2	3B	✓		0.7218
	Llama-3.3	70B	✓		0.7887
Qwen	Qwen2.5	0.5B			0.5396
	Qwen2.5	1.5B			0.6928
	Qwen2.5	1.5B	✓		0.6972
	Qwen2.5	3B			0.7632
	Qwen2.5	7B			0.8055
	Qwen2.5	7B	✓		0.7967
	Qwen2.5	14B			0.8231
	Qwen2.5	32B			0.8081
	Qwen2.5	32B	✓		0.8247
	Qwen2.5	72B			0.7201
	Qwen2.5	72B	✓		0.8257
	QwQ	32B		✓	0.7614
	Qwen3	1.7B			0.6937
	Qwen3	4B			0.7905
	Qwen3	8B			0.6708
Agent	**MoCA-Agent (ours)**	27B			**0.8688**

Table B.8: ESGenius accuracy under the RAG setting. Non-MoCA-Agent rows are open-source baselines reproduced from He et al. (2025); MoCA-Agent passes each question's released supporting passage into the agent's context (oracle-RAG-equivalent, directly comparable to the open-source RAG column). Proprietary API rows (GPT-4o, o3, o4-mini, DeepSeek-R1/V3, Qwen2.5-Max) from the original paper are omitted because their RAG numbers were not reported. **IT**: instruction-tuned; **Rea**: reasoning-focused; ✓= yes, blank = no.

plagues plain PoT (program runs but its printed output is not the final answer) has been almost fully eliminated. Third, mean market confidence $\bar{\Gamma}$ varies meaningfully by category: it is highest on ESGenius (0.465), where claims are factual/single-step and the panel agrees easily, lowest on FinanceMath (0.220), where each problem mixes several formulas and the SKEPTIC role frequently shorts the formula claim.

C Qualitative Example: MoCA-Agent on FinQA

To illustrate how the trader market actually drives the final answer, we walk through one representative FinQA example end-to-end. The example is

Dataset	N	Exec.%	Cons.%	$\bar{\Gamma}$
FinQA	1,147	98.43	98.43	0.323
DM-Simplong	100	96.00	96.00	0.223
DM-Complong	300	98.00	98.00	0.345
FinanceMath (val)	200	98.00	98.00	0.220
HiTab	1,584	98.99	98.99	0.237
MultiHiertt (dev)	1,044	98.37	98.37	0.257
TabMWP	1,000	100.00	100.00	0.258
WTQ	4,344	99.36	99.36	0.347
ESGenius	1,136	100.00	100.00	0.465
FinChart QA	2,284	99.65	99.65	0.139

Table B.9: Per-dataset internal diagnostics for MoCA-Agent. **Exec.%**: fraction of programs that ran without error. **Cons.%**: fraction of programs whose printed output matched the candidate's stated answer. $\bar{\Gamma}$: mean market confidence over cited claims (Eq. 7). A high execution rate paired with a moderate $\bar{\Gamma}$ is the expected operating regime: the verifier admits a candidate when the market is decisive enough on the *cited* subset of claims, not on the full catalog.

taken verbatim from our run on the FinQA test split (record `AES/2010/page_227.pdf-1`) and is one of many cases where the market visibly suppresses an over-confident but wrong candidate fact before it can enter the program.

Question. *"What percentage of recourse debt as of December 31, 2010 matures after 2015?"* The gold answer is 0.68343; MoCA-Agent returns 0.6834345186470078. The question is classified by the catalog builder as *percentage_change*.

Claim catalog. Nine atomic claims are extracted from the question and table (Table C.1). C1 through C7 are typed as *fact* claims (one per cell needed to answer), C8 as a *formula* claim, and C9 as a candidate answer interval supplied by the catalog builder for a sanity check.

Specialist trading. The four trader roles independently price each claim. Three roles short C3 and C4 because the maturity-year column has slipped (both 2012 and 2013 are listed as 2,014, the same number that would naturally appear as a *year*). The EXTRACTOR writes *"The claim incorrectly interprets the table value '2014' for the maturity column"*; the ACCOUNTANT writes *"The table lists the maturity amount for 2012 as '2014', which is arithmetically inconsistent"*; the SKEPTIC writes *"Identical suspicious value to C3; if these are typos for smaller amounts the totals don't reconcile"*. The remaining four roles all buy C1, C7, C8, and

ID	Kind	Summary	Value
C1	fact	Total recourse debt as of Dec 31, 2010	4,612
C2	fact	Recourse debt maturing in 2011	463
C3	fact	Recourse debt maturing in 2012	2,014
C4	fact	Recourse debt maturing in 2013	2,014
C5	fact	Recourse debt maturing in 2014	497
C6	fact	Recourse debt maturing in 2015	500
C7	fact	Recourse debt maturing after 2015	3,152
C8	formula	Percentage = (C7 / C1) $\times 100$	–
C9	answer-interval	Percentage is in [68, 69]	–

Table C.1: Claim catalog for the FinQA *recourse debt* example. Note that C3 and C4 share the suspicious value 2,014, exposing a likely OCR / column-shift artefact in the original table.

C9 at high prices.

Market clearing. Clearing the order books yields the snapshot in Table C.2. C1, C7, C8, and C9 are accepted at prices ≥ 0.70; C3 and C4 are flatly rejected with clearing price 0; C2 and C6 are also rejected (irrelevant to the "after 2015" numerator); C5 is uncertain. Crucially, the synthesizer is now forbidden from citing C3 or C4 even though both numbers appear in the table. A vanilla program-of-thought baseline would have happily summed them into the numerator.

ID	Status	Price	Confidence
C7	accepted	1.00	1.00
C8	accepted	1.00	1.00
C9	accepted	1.00	0.99
C1	accepted	0.70	0.40
C5	uncertain	0.50	0.00
C2	rejected	0.00	1.00
C3	rejected	0.00	1.00
C4	rejected	0.00	1.00
C6	rejected	0.00	1.00

Table C.2: Cleared market for the FinQA *recourse debt* example. Eight of nine claims clear with confidence ≥ 0.99; only C5 (the 2014 maturity) remains uncertain and is unused by the final program.

Synthesized program. With only the accepted claims available, the synthesizer emits the following Python:

```
# claims: C1, C7, C8
total_debt = 4612          # C1
debt_after_2015 = 3152     # C7
percentage = debt_after_2015 / total_debt  # C8
print(percentage)
```

The code executes cleanly and prints 0.6834345186470078, which matches the gold answer 0.68343 after FinQA-style rounding.

Why this example matters. The two rejected fact claims C3 and C4 (both = 2,014) are exactly the kind of silent error a vanilla PoT agent commits: the LLM reads the column-shifted number off the table and writes it into the program without ever flagging the inconsistency. MoCA-Agent catches it because three different trader roles, each looking at the same table through a different inductive bias (extraction, accounting, skeptic), independently short the suspicious claim, and the market mechanism aggregates that disagreement into an explicit `rejected` verdict that the synthesizer is structurally prevented from overriding.

D Prompt Library

The agent pipeline has **nine** distinct LLM calls, each driven by its own system prompt; for the trader panel, the four roles share the same JSON schema but carry different inductive biases (extraction vs. formula vs. accounting vs. skepticism), so we show each role separately. All prompts are byte-identical across the ten datasets used in the main paper; only the user-side payload (question, table, context, claim catalog, market snapshot, executed candidates) changes between calls.

The nine prompts are organized into four groups: **(a) Claim construction** (§D.1), **(b) Specialist trading** (§D.2), **(c) Code synthesis and repair** (§D.3, §D.4), and **(d) Selection and arbitration** (§D.5, §D.6). A control-flow diagram in Figure 1 shows how these calls connect; the table below maps each call to its temperature and maximum output length under our default vLLM configuration.

D.1 Claim Catalog Builder

The catalog builder is the first LLM call in the pipeline. It reads the question, table, and free-form context, and emits a small JSON catalog of typed atomic claims that the rest of the pipeline operates on. The catalog is the only mechanism by which the question's information needs become tradable; every later stage is structurally prevented from citing facts that are not in the catalog.

MoCA-Agent claim-catalog builder prompt

System Prompt: You are the claim-catalog builder for an internal financial prediction market.

Your job is to transform a financial question into a small catalog of tradable atomic claims.

Output JSON only with this schema:

```
{
  "problem_type": "percentage_change|ratio|sum|
                   difference|average|comparison|other",
  "claims": [
    {
      "id": "C1",
      "kind": "fact|formula|direction|unit|sign|
               answer_interval",
      "summary": "short plain-English claim",
      "value":   "canonical value for the claim",
      "evidence":"short exact quote from the
                  table/context when applicable",
      "metadata":{"period":"...","unit":"...","note":"..."}
    }
  ]
}
```

Rules:

- Create sequential IDs `C1, C2, ...`.
- Include 4 to 10 claims total.
- Prefer atomic, contestable claims instead of long explanations.
- Include competing alternatives when ambiguity exists.
- Include formula and direction claims when they matter.
- Include an `answer_interval` claim only if you can bound the answer reasonably.
- Do not solve the whole problem in prose.

User Prompt (template):
```
Question:\n{question}
Table:\n{table or "(no table provided)"}
Context:\n{context or "(no additional context)"}
Maximum claims: {max_claims}
```

Figure D.1: The claim-catalog builder prompt. Output is a strict JSON object that downstream stages parse; the `kind` field types each claim (fact / formula / direction / unit / sign / answer-interval) and the optional `evidence` quote anchors a fact claim to a specific cell of the table.

Stage	Temp.	Max tokens
Catalog builder	0.0	2048
Trader (per role)	0.0	2048
Synthesizer	0.1	2048
Repair	0.0	2048
Selector (committee)	0.0	2048
Arbiter	0.0	2048

Table D.1: Decoding settings per LLM call in the MoCA-Agent pipeline.

D.2 Specialist Trader Panel

The four trader roles (EXTRACTOR, FORMULA, ACCOUNTANT, SKEPTIC) share an output schema: each returns a JSON `trades` list of `buy`/`sell` orders against claim IDs from the catalog, but their system prompts encode different inductive biases. The four prompts are reproduced verbatim in Figures D.2 to D.5. The differences between them are deliberately narrow: each role concentrates its weighted volume on a distinct failure mode (cell selection vs. formula family vs. unit / sign consistency vs. adversarial shorting of weak claims), and the market clearing rule (Equations 3 to 5 of the main paper) aggregates their disagreement into a single price.

D.3 Synthesizer

After the market clears, the synthesizer writes the final Python program. Crucially, the synthesizer prompt is the first stage in the pipeline that sees *both* the catalog and the cleared market snapshot. The trader books are not exposed. The synthesizer is also the stage where structural rules (operation

MoCA-Agent extractor trader prompt

System Prompt: You are the extractor trader in a financial claim market.

Focus on:

- which values are actually relevant
- exact row / column / year selection
- whether a quoted evidence string really supports a fact claim

You trade heavily on *fact* claims and may also trade on *unit* or *direction* claims.
Output JSON only:

```
{
  "role": "extractor",
  "trades": [
    {
      "claim_id": "C1",
      "side": "buy|sell",
      "size": 1,
      "price": 0.75,
      "reason": "one-sentence justification"
    }
  ],
  "risk_note": "optional short note"
}
```

User Prompt (template): `Question, Table, Context, ClaimCatalog` are passed verbatim; the trader sees only the catalog (not free-form text from other roles) to keep books independent.

Figure D.2: Extractor trader prompt. This role over-weights fact claims and quotes the relevant evidence string from the table when explaining a buy. In the worked example of §C the extractor wrote *"The table explicitly lists 'total recourse debt' as $4,612"* when buying C1.

must match the question class, percent answers as decimals, yes/no questions get yes/no answers, no tuples/lists/booleans) are enforced via the system prompt, with the verifier later checking compliance deterministically.

D.4 Code-Aware Repair

The repair prompt is invoked at most once per question, only when the verifier flags the first program as invalid. The prompt receives the runtime error *and* the verifier's structural reason list (e.g. *missing_formula_or_unit_claim*, *percent_scaled_by_100*), so the repair stage can target the specific defect rather than rewrite the program blindly.

D.5 Hybrid Selector Committee

When both a MoCA-Agent candidate and a baseline-PoT candidate are available, a four-aspect committee (lenses: GENERAL, EXTRACTION, FORMULA, SCALE) compares the two side by side. Each lens is the same system prompt with one extra "focus on . . . " sentence prepended; votes are then aggregated with a confidence-weighted threshold (§3.5).

D.6 Conflict Arbiter

If the selector committee splits and the two candidates' final outputs disagree, a single arbiter LLM call synthesizes a corrected third program. The arbiter is allowed to follow either candidate or to write a fresh program, but is structurally constrained to ground itself in the same claim catalog used by both candidates.

MoCA-Agent formula trader prompt

System Prompt: You are the formula trader in a financial claim market.

Focus on:

- operation family
- denominator choice
- direction of percentage change
- sign conventions for rates, deltas, and comparisons

Output JSON only:

```
{
  "role": "formula",
  "trades": [
    {
      "claim_id": "C1",
      "side": "buy|sell",
      "size": 1,
      "price": 0.75,
      "reason": "one-sentence justification"
    }
  ],
  "risk_note": "optional short note"
}
```

Figure D.3: Formula trader prompt. This role concentrates on the *formula* and *direction* claims, and on whether the chosen denominator is internally consistent with the question phrasing.

MoCA-Agent accountant trader prompt

System Prompt: You are the accountant trader in a financial claim market.

Focus on:

- unit consistency
- sign handling
- whether a claim mixes incompatible measures or periods
- whether the interpretation is financially coherent

You are explicitly encouraged to *short* claims that look numerically or semantically inconsistent.
Output JSON only:

```
{
  "role": "accountant",
  "trades": [...],
  "risk_note": "optional short note"
}
```

Figure D.4: Accountant trader prompt. The accountant role has the highest role weight ($w_{\text{ACCOUNTANT}} = 1.10$) and is the role responsible for shorting the column-shift artefact in the worked example: *"The table lists the maturity amount for 2012 as '2014', which is arithmetically inconsistent"*.

MoCA-Agent skeptic trader prompt

System Prompt: You are the skeptic trader in a financial claim market.

Your job is to find likely failure modes:

- wrong year ordering
- wrong denominator
- wrong unit scale
- wrong sign
- unsupported evidence

Be *aggressive* about selling weak claims.
Output JSON only:

```
{
  "role": "skeptic",
  "trades": [...],
  "risk_note": "optional short note"
}
```

Figure D.5: Skeptic trader prompt. The skeptic carries a slightly *lower* role weight ($w_{\text{SKEPTIC}} = 0.95$) so its adversarial bias does not overwhelm the market; its role is to prevent confidently-stated but wrong claims from clearing as accepted.

MoCA-Agent synthesizer prompt

System Prompt: You are the MoCA-Agent synthesizer.

You receive a claim catalog and a cleared market snapshot. Your task is to write the final executable program.
Output JSON only:

```
{
  "reasoning":      "short explanation of which claims you trust",
  "used_claim_ids": ["C1", "C4"],
  "abstain":        false,
  "abstain_reason": "",
  "code":           "python code",
  "answer":         "final answer"
}
```

Rules:

- Prefer accepted claims; avoid rejected ones unless absolutely necessary. If the market is too conflicted, abstain.
- Use only basic Python. No pandas, no file access.
- The last line of code must `print` exactly one final value.
- Add short comments that cite claim IDs, e.g. `# claims: C1, C4`.
- `used_claim_ids` must list every claim actually used.
- Use at least one fact claim; for ratio / percent / comparison / average / sum / difference questions, use at least two.
- For ratio / percent / rate questions, also use at least one formula, unit, direction, or sign claim when available.
- Honor explicit task metadata (answer type, scale) over generic defaults.
- For percentage-like questions, print the *decimal* fraction (e.g. FinQA convention). Do *not* multiply by 100.
- For yes/no questions, print only "yes" or "no". Never print Python booleans `True` or `False`.
- Never print tuples, lists, or dictionaries.

Output (worked example, FinQA *recourse debt*):

```
{
  "reasoning":      "Market accepted C1, C7, C8; ...",
  "used_claim_ids": ["C1", "C7", "C8"],
  "abstain":        false,
  "code": "# claims: C1, C7, C8\ntotal_debt = 4612 ...",
  "answer":         "0.6834345186470078"
}
```

Figure D.6: Synthesizer prompt. The verifier (§3.5) then checks every rule mechanically after execution; any structural violation triggers a single market-aware repair round.

MoCA-Agent market-aware repair prompt

System Prompt: You are the MoCA-Agent repair synthesizer.

The previous program failed or looked inconsistent. Repair it using the market snapshot.
Output JSON (same schema as synthesizer):

```
{
  "reasoning": "...",
  "used_claim_ids": ["C1"],
  "abstain": false,
  "abstain_reason": "",
  "code": "python code",
  "answer": "final answer"
}
```

Rules:

- Do not invent new evidence.
- Prefer the highest-confidence accepted claims.
- If market support is too weak to repair safely, abstain.
- Repair any structural issues: missing fact claims, missing formula/unit support, boolean outputs, tuple outputs, or percentage answers scaled by 100.
- Honor explicit task metadata over generic defaults.
- For yes/no questions, print only "yes" or "no".
- For percentage-like questions, print the decimal fraction, not the percentage-point form.

User Prompt (template): the original question / table / context / catalog, plus the previous program, its execution error, its output, and the verifier's structural reason list.

Figure D.7: Market-aware repair prompt. Repair is structurally bounded to one round; if the second attempt still fails the verifier the selector committee falls back to the baseline proposer.

MOCA-AGENT selector committee prompt

System Prompt: You are the final selector for MOCA-AGENT.

You must choose between two already-executed candidates:

- `baseline`: the existing FinCodeAgent program-of-thought.
- `moca`: the market-of-claims candidate.

Output JSON only:

```
{
  "choice":     "baseline|moca|tie",
  "confidence": 0.0,
  "rationale":  "short explanation"
}
```

Decision rules:

- Be conservative. If uncertain, choose `baseline`.
- Focus on exact value extraction, denominator choice, sign, unit scale, and yes/no semantics.
- Honor explicit task metadata when present.
- For percentage-like questions, prefer decimal fractions.
- For yes/no questions, prefer candidates that truly answer yes/no with the correct comparison direction.
- Use the market snapshot as evidence, but do not trust it blindly.

Lens add-ons (one sentence prepended to the system prompt):

- GENERAL: "Take a balanced view over extraction, arithmetic, sign, and unit handling."
- EXTRACTION: "Prioritize exact row, year, and evidence selection."
- FORMULA: "Prioritize arithmetic family, denominator, direction, and comparison semantics."
- SCALE: "Prioritize unit scale, totals vs. components, and benchmark answer-format correctness."

Figure D.8: Selector-committee prompt. The four lenses act as independent voters; a candidate is promoted only if the confidence-weighted margin exceeds 0.65 (default).

MoCA-Agent conflict-arbiter prompt

System Prompt: You are the disagreement arbiter for MoCA-Agent.

Two executed candidates disagree. Produce the best corrected final program.
Output JSON only:

```
{
  "reasoning":      "short explanation of which claims or
                    candidate pieces you trust",
  "used_claim_ids": ["C1", "C4"],
  "abstain":        false,
  "abstain_reason": "",
  "code":           "python code",
  "answer":         "final answer"
}
```

Rules:

- You may follow the baseline candidate, follow the MoCA candidate, or synthesize a corrected third program.
- Do not invent evidence; ground yourself in the question, table, context, and claim catalog.
- Prefer exact value extraction, correct denominator, correct sign, and correct unit scaling.
- The last line of code must print exactly one final value.
- Honor explicit task metadata; for percentage-like questions, print decimals; for yes/no, print only "yes"/"no".
- Never print tuples, lists, or dictionaries.
- If the evidence is genuinely too weak, abstain.

Figure D.9: Conflict-arbiter prompt. This is the only LLM call in the pipeline that sees both candidate programs side by side; it is gated by the selector committee and fires on $< 10\%$ of inputs at the default thresholds.